



设计模式

面向嵌入式软件开发



针对C语言及嵌入式软件开发的
伴读教程，陪伴学习设计模式这一
进阶知识点。

嵌入式视角的 GoF 设计模式实践

基于《Design Patterns: Elements of Reusable Object-Oriented Software》
从嵌入式系统架构出发，系统讲解 GoF 23 种设计模式，并提供工程级代码示例。

项目简介

本仓库围绕 **GoF 设计模式原书结构**展开讲解，结合嵌入式系统开发场景进行重构与落地实现。

不同于面向 Web / 业务系统的讲解方式，本项目强调：

- 面向嵌入式系统的架构设计思维
- 资源受限环境下的模式取舍
- 模块解耦与可扩展性
- 产品线与平台演进能力
- 工程化可落地的代码结构

适合：

- 嵌入式开发工程师
- C 工程师
- 系统架构设计人员
- 希望系统掌握 GoF 理论与工程实践的开发者

为什么嵌入式更需要设计模式？

嵌入式系统通常具有：

- 生命周期长
- 平台迁移频繁
- 硬件耦合严重
- 可维护性要求高

设计模式在嵌入式中的核心价值：

- 抽象硬件差异
- 降低驱动与业务耦合
- 支持产品线扩展
- 提升系统可测试性
- 改善架构演进能力

在以下场景尤为重要：

- BSP 分层设计
- 驱动抽象层
- 中间件架构
- 多平台产品架构

支持作者



 [点击进入 B 站视频合集](#)

如果这个项目对你有帮助，我这里想直接说一句：

 请帮我去 B 站点个赞 、投个币 ，有条件的话冲个电  支持一下

这对我真的很重要。

我做这个项目投入了大量时间，把 GoF 设计模式结合嵌入式完整拆解、写代码、做讲解，本质上是在做一套长期的体系内容。

但这些内容能不能持续更新，很大程度取决于有没有正反馈。

你一个小动作的意义：

-  点赞 → 让更多人看到这个系列
-  投币 → 直接支持内容持续产出
-  充电 → 让我有动力把这套体系完整做完



设计模式

面向嵌入式软件开发



针对C语言及嵌入式软件开发的
伴读教程，陪伴学习设计模式这一
进阶知识点。

1.4 设计模式的编目

抽象工厂模式 (Abstract Factory Pattern / 抽象工厂 / 工厂族模式)：

提供一个创建一系列相关或相互依赖对象的接口，而无需指定它们的具体类型；在嵌入式C中可用于**多种通信接口族的封装**，例如 `comm_factory_create()` 根据配置创建出一组匹配的驱动：UART、SPI、I2C 的收发与初始化函数集合，使上层通信模块在切换总线类型时不需修改业务逻辑，只替换工厂即可。

适配器模式 (Adapter Pattern / 适配器 / 转接器模式)： 适配薄层 - 移植常用

将一个类的接口转换成客户端期望的另一种接口，使原本不兼容的模块可以协同工作；在嵌入式 C 中常用于**统一不同外设驱动的接口格式**，例如将一个只能用 `HAL_UART_Transmit()` 的旧驱动包装成通用的 `comm_send()` 接口，使其能被系统的通信框架直接调用。

桥接模式 (Bridge Pattern / 桥接模式 / 桥梁模式)： LVGL的打点函数

将抽象部分与其实现部分分离，使它们可以独立变化；在嵌入式 C 中常用于**分离设备逻辑与硬件驱动实现**，例如一个“显示控制模块”可独立于具体的显示接口（SPI、RGB、HDMI）而存在，只需在运行时“桥接”不同驱动实现。

建造者模式 (Builder Pattern / 生成器模式 / 构造者模式)： TCP通信 -分步骤 或者 事件状态机

将一个复杂对象的构建过程与它的表示分离，使同样的构建步骤可以创建不同的对象；在嵌入式 C 中常用于**分阶段初始化复杂外设或配置结构体**，例如通过分步设置参数后统一生成一个完整的通信配置或任务对象。

初始化网卡硬件 (MAC)

初始化物理层 (PHY)

配置 IP 地址、子网、网关

初始化 TCP/IP 协议栈

注册回调、启动任务

责任链模式 (Chain of Responsibility Pattern / 责任链 / 职责链模式): 串行分层执行

将请求沿着处理者链传递,直到有对象处理它,从而避免请求发送者与接收者耦合;在嵌入式 C 中常用于**事件或消息处理流程的分发**,例如一个按键事件或通信消息依次通过不同处理模块(滤波器、解码器、业务处理)处理,直到某个模块处理完毕。

命令模式 (Command Pattern / 命令模式 / 指令模式): 消息处理队列-指令对象化

将一个请求封装成对象,从而让你可用不同的请求、队列或日志参数化请求,以及支持撤销操作;在嵌入式 C 中常用于**统一控制硬件操作或任务指令**,例如按键操作、外设控制、定时任务调度都可以封装成命令对象,由上层统一调用或排队执行。

组合模式 (Composite Pattern / 组合模式 / 部件-整体模式): 文件系统

将对象组合成树形结构以表示“部分-整体”的层次结构,使客户端可以统一对待单个对象和组合对象;在嵌入式 C 中常用于**菜单、UI控件、文件系统或任务集合管理**,例如 LVGL 的容器控件可以包含按钮、标签、滑块,操作统一通过父容器处理。

装饰者模式 (Decorator Pattern / 装饰模式 / 装饰器模式): ulog打印

动态地给对象添加额外功能,而不改变其原有结构或接口;在嵌入式 C 中常用于**给驱动或任务增加功能**,例如在串口发送函数外层增加日志记录、校验、加密等功能,而不修改原始驱动代码。

外观模式 (Facade Pattern / 外观模式 / 门面模式): 和linux驱动设计原则有冲突; 适合多人协作,实现功能逻辑块

为子系统提供一个统一的高层接口,使子系统更易使用,降低依赖复杂性;在嵌入式 C 中常用于**封装多个外设或驱动操作成统一接口**,例如对一个完整通信模块(UART + DMA + CRC + 事件回调)提供一个简单的 `comm_init()`、`comm_send()` 接口,让上层业务不关心内部复杂细节。

工厂方法模式 (Factory Method Pattern / 工厂方法 / 虚拟构造器模式): 这个模式和抽象工厂模式区别? 那个更灵活

定义一个创建对象的接口,让子类决定实例化哪一个类,从而使一个类的实例化延迟到子类;在嵌入式 C 中常用于**根据具体硬件或外设类型生成相应驱动实例**,例如不同型号的传感器或通信接口由不同的工厂函数创建,但上层调用统一接口获取对象。

Abstract Factory: 上层只管调用统一接口,内部逻辑决定具体硬件; Factory Method: 上层逻辑决定使用哪一个具体硬件对象。

享元模式 (Flyweight Pattern / 享元模式 / 轻量级模式): 通信的接收缓冲区 ; 指令队列

通过共享尽量多的相同对象来减少内存使用, 把对象的状态分为**可共享内部状态**和**不可共享外部状态**; 在嵌入式 C 中常用于**大量相似对象的管理**, 例如 GUI 中重复的控件样式、字体、图标, 或者通信协议中大量重复配置的数据结构。

解释器模式 (Interpreter Pattern / 解释器模式): 自己写的字符对比 二进制对比 ; 大一点 python解释器 lua解释器

给定一种语言, 定义它的文法表示, 并定义一个解释器来解释句子; 在嵌入式 C 中常用于**协议解析、命令解析或脚本解释**, 例如解析简单通信协议、AT 命令或自定义脚本指令。

迭代器模式 (Iterator Pattern / 迭代器模式 / 游标模式): 一组IO中 , 哪些为高? 一堆数据, 几种状态 , 返回其中某种状态的 数据序号

提供一种方法顺序访问一个聚合对象中的各个元素, 而又不暴露该对象的内部表示; 在嵌入式 C 中常用于**遍历数组、链表或任务队列**, 例如遍历消息队列、传感器数据列表或 GUI 控件集合。

FlashDB KV数据库

中介者模式 (Mediator Pattern / 中介者模式 / 调停者模式): 消息总线 ; 共享内存 openAMP

用一个中介对象来封装对象之间的交互, 使各对象不需要直接引用彼此, 从而降低耦合; 在嵌入式 C 中常用于**模块间消息或事件通信**, 例如不同驱动或任务通过中介者发送消息, 而不直接调用对方接口。

备忘录模式 (Memento Pattern / 备忘录模式 / 快照模式): ???

在不暴露对象实现细节的情况下, 捕获并保存对象的内部状态, 以便将来恢复; 在嵌入式 C 中常用于**配置快照、状态保存或回滚机制**, 例如保存外设寄存器配置、任务状态或通信会话状态, 方便恢复。

RTOS 的线程上下文切换就是 Memento 模式: 保存任务的完整 CPU 状态快照, 当切换回来时恢复, 保证任务无缝执行”。

backtrace

观察者模式 (Observer Pattern / 观察者模式 / 发布-订阅模式): 一对多, 单触发多执行; 消息订阅

定义对象间的一种一对多依赖, 当一个对象状态发生变化时, 所有依赖它的对象都会得到通知并自动更新; 在嵌入式 C 中常用于**事件驱动、消息通知或状态同步**, 例如传感器状态变化通知多个任务、驱动事件广播、GUI 控件更新。

原型模式 (Prototype Pattern / 原型模式 / 克隆模式): 创建相似任务或配置对象 ; UI里面常见

通过复制现有对象来创建新对象，而不是通过构造函数创建；在嵌入式 C 中常用于**快速生成相似配置对象或任务实例**，例如重复初始化配置、创建相似通信帧或 GUI 控件实例。

代理模式 (Proxy Pattern / 代理模式 / 代理类模式)： 非直接读写寄存器 通过结构体对象 函数调用间接访问

为另一个对象提供一个**代理或占位符**以控制对它的访问；在嵌入式 C 中常用于**外设访问控制、延迟初始化或安全访问**，例如对外设寄存器操作做统一接口、懒加载硬件资源或通过中间层控制访问。

单例模式 (Singleton Pattern / 单例模式 / 全局唯一模式)： 全局变量 全局对象 ??? 多线程 得加锁

保证一个类只有一个实例，并提供全局访问点；在嵌入式 C 中常用于**全局硬件资源管理、系统配置或全局状态对象**，例如系统日志模块、全局通信接口或外设管理器。

状态模式 (State Pattern / 状态模式 / 状态机模式)： 按钮的 点击 双击 长按 这种状态切换

允许对象在内部状态改变时改变它的行为，看起来像修改了对象的类；在嵌入式 C 中常用于**事件驱动的状态机、任务或外设状态管理**，例如设备工作模式切换、通信协议状态机、按键处理。

策略模式 (Strategy Pattern / 策略模式 / 算法封装模式)： 定义统一的数据输入输出接口

定义一系列算法，将每个算法封装起来，并使它们可以互换；在嵌入式 C 中常用于**不同功能或算法的可替换实现**，例如不同滤波器、数据压缩算法或通信协议处理方式。

模板方法模式 (Template Method Pattern / 模板方法模式 / 固定流程模式)： 流程可复用，步骤可定制

在父类中定义一个算法的骨架，而将某些步骤延迟到子类实现；在嵌入式 C 中常用于**固定流程但可定制步骤的任务或外设操作**，例如初始化外设、通信序列或任务执行流程。

协议转换模块中，固定收→解析→发的流程属于 Template Method，不同协议的解析和转发逻辑是可替换步骤

访问者模式 (Visitor Pattern / 访问者模式 / 元素操作分离模式)： 对不同的对象 附加统一的新操作 ； RTT 外设使用计数

将作用于某对象结构的操作封装成独立的访问者，使得可以在不改变对象结构的前提下增加新操作；在嵌入式 C 中常用于**不同数据类型或协议帧的统一处理**，例如对多种外设数据包或消息结构做统一解析、统计或日志处理。

1.5 组织编目

1. 创建型模式与对象的创建有关；结构型模式处理类或对象的组合；行为型模式对类或对象怎样交互和怎样分配职责 进行描述。
2. 类模式处理类和子类之间的关系，这些关系通过继承建立，是静态的，在编译时便确定下来了。对象模式处理对象间的关系，这些关系在运行时是可以变化的，更具动态性。

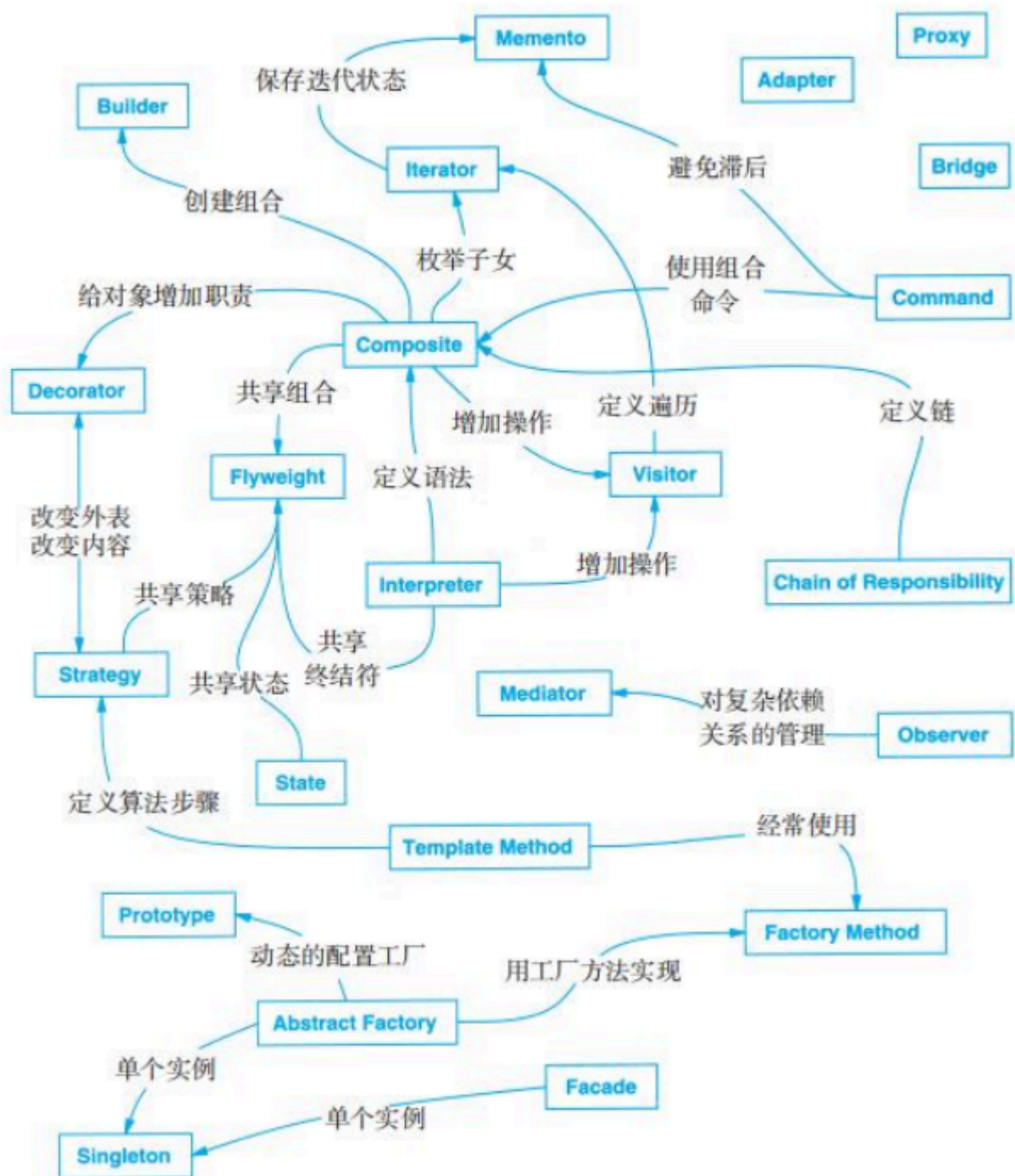
		目 的		
		创 建 型	结 构 型	行 为 型
范围	类	Factory Method(3.3)	Adapter(类)(4.1)	Interpreter(5.3) Template Method(5.10)
	对象	Abstract Factory(3.1) Builder(3.2) Prototype(3.4) Singleton(3.5)	Adapter(对象)(4.1) Bridge(4.2) Composite(4.3) Decorator(4.4) Facade(4.5) Flyweight(4.6) Proxy(4.7)	Chain of Responsibility(5.1) Command(5.2) Iterator(5.4) Mediator(5.5) Memento(5.6) Observer(5.7) State(5.8) Strategy(5.9) Visitor(5.10)

创建型类模式将对象的部分创建工作延迟到子类，而创建型对象模式则将它延迟到另一个对象中。

结构型类模式使用继承机制来组合类，而结构型对象模式则描述了对对象的组装方式。

行为型类模式使用继承 描述算法和控制流，而行为型对象模式则描述了一组对象怎样协作完成单个对象所无法完成的任务。

根据模式的“相关模式”部分所描述的它们怎样互相引用来组织设计模式。





设计模式

面向嵌入式软件开发



针对C语言及嵌入式软件开发的
伴读教程，陪伴学习设计模式这一
进阶知识点。

1.6 设计模式怎样解决设计问题

1.6.1 寻找合适的对象

1 设计模式的“寻找合适的对象”核心思想

原书中这一节主要意思是：在设计系统时，你要清楚 哪个对象应该承担某个职责，也就是 责任归属。

- 不是把所有功能都堆到一个对象上，也不是随意分配职责。
- 要考虑对象的 自然性（自然地承担某些行为）和 封装性。

2 嵌入式 C 的特点

在嵌入式系统里，我们常见的特点：

- 资源有限：内存小、CPU慢。
- 模块化强：代码通常按功能模块划分，硬件驱动、应用逻辑、通信协议分开。
- 硬件绑定紧密：对象通常与外设或数据结构一一对应。
- 面向结构而非对象：C 没有类和继承，更多用 `struct` + 函数指针来模拟对象行为。

所以，我们在嵌入式 C 里解读“寻找合适的对象”时，更多是 寻找合适的结构体/模块承担职责。

3 如何在嵌入式 C 中寻找合适对象

可以按以下思路：

(1) 按职责划分结构体

- 每个模块对应一个“对象”，封装它的状态和行为。

(2) 模块自然性原则

- 谁最自然处理这件事，就把职责给谁。
- 例如：
 - UART 数据收发 → UART 驱动模块。
 - LED 灯闪烁 → LED 模块。
- 避免把 LED 闪烁逻辑放到 UI 模块里，这就是“职责不自然”。

(3) 接口与回调模拟多态

- C 没有类，但函数指针可以让模块暴露行为。

(4) 考虑硬件约束

- 一个对象不应该跨越多个硬件模块职责。
- 例如：
 - I2C 总线管理 → I2C 控制器对象
 - 每个传感器 → 传感器对象
- 避免一个对象直接操作 SPI、UART、GPIO 混乱。

假设我们做一个 小型风扇控制系统：

- 按键控制风扇开关和转速。
- 电机驱动负责风扇运转。
- LED 指示灯显示风扇状态（开/关/转速等级）。
- 我们要在 C 里设计模块，让每个模块承担最自然的职责。

(1) 按键对象

职责：负责读取按键状态、去抖动、发送事件。

(2) 电机对象

职责：控制风扇开关与转速

(3) LED 指示对象

职责：显示风扇状态

主程序：负责模块间的协调，而不是直接去操作 GPIO 或寄存器。

这样每个对象都有 **自然职责**，符合设计模式里“寻找合适对象”的思想。

总结 道法自然，不要强求。

1. **模块化结构体设计：**每个 `struct` 对应自然承担职责的功能块。
2. **封装行为与状态：**用函数指针封装方法，实现“对象化”。
3. **职责自然分配：**把功能交给最“合理”的模块，避免耦合过高。
4. **遵守硬件边界：**对象不跨越多个硬件域，便于维护和扩展。

1.6.2 决定对象的粒度

嵌入式里面怎么去区分项目内的对象颗粒度，这个事情比较困难。

2 具体模式解析

1. Facade（外观模式，4.5）

- 描述如何用 **一个对象表示完整子系统**。
- 嵌入式理解：比如把风扇系统的“按键+电机+LED”封装成一个 `FanController` 外观对象，让主程序只调用一个接口即可操作整个系统。
- 特点：粒度偏大，职责集中，简化调用。

2. Flyweight（享元模式，4.6）

- 描述如何支持 **大量最小粒度的对象**，共享公共状态。
- 嵌入式理解：比如大量相同 LED 或传感器对象，内部共享寄存器映射表或配置数据，避免重复开销。
- 特点：粒度偏小，但通过共享管理大量对象。

3. 其他将对象分解成小对象的模式

- 例如组合模式（Composite）、策略模式（Strategy）等
- 让一个大对象被拆分成若干小对象，每个小对象专注某个职责。
- 嵌入式理解：一个复杂外设（如 UART 模块）可以拆成 TX、RX、Buffer 等小对象，各自处理对应行为。

4. Abstract Factory（抽象工厂，3.1） & Builder（建造者，3.2）

- 产生 **专门负责生成其他对象的对象**。

- 嵌入式理解：比如有一个 `PeripheralFactory`，根据不同硬件生成 UART、SPI、I2C 对象，方便扩展和复用。

5. Visitor（访问者，5.10） & Command（命令，5.2）

- 生成的对象专门 负责对其他对象或对象组发起请求。
- 嵌入式理解：比如有一个 `Command` 对象集合，用于批量操作风扇、LED、传感器等，每个对象只关注自己的行为，实现解耦。

3 嵌入式 C 的启示

- **粒度控制**：设计模式告诉我们有对象（Facade）、小对象（Flyweight）、生成对象（Factory/Builder）等策略。
- **职责清晰**：每个对象都有“自然职责”，不要乱堆功能。
- **管理大量对象**：Flyweight、Command、Visitor 等模式提供了管理小对象或批量操作的方法。
- **适配硬件约束**：通过模式可以在嵌入式资源有限的情况下，实现模块化、可复用、易维护的对象设计。

同样的项目，不同的团队或者个人，使用的对象颗粒度就是不一样。

场景：

- 功能：按钮按下控制 LED
- 新增需求：长按不亮，短按亮，双击闪烁，长按呼吸灯

不同团队设计的对象颗粒度：

团队人数	对象设计	特点	新需求修改难度
1人	按钮检测电平 + 点灯逻辑在一个模块	粒度粗	每次新增功能都要改同一模块，改动大，容易出错
2人	按钮+LED 驱动模块，逻辑处理模块	粒度中	修改逻辑处理模块即可，驱动模块无需改动，修改局部，风险小
3人	按键驱动、LED驱动、逻辑处理模块	粒度细	新需求只修改逻辑处理模块或增加新对象，最清晰，速度快，扩展性高，但对象数量多，需要团队协作

说个最简单 但是又很抽象的例子

一个按钮点一个led，按钮按下点亮led

1个人，检测电平->点灯

2个人，按钮和led 驱动；逻辑处理

3个人，按键驱动；LED驱动：逻辑处理

这个新增需求，要求长按灯不亮，短按才亮，1个人的团队就得更改比较多； 2.3人团队就一个修改按钮驱动 一个新增处理逻辑

又新增需求 双击按钮 灯闪烁，长按 灯呼吸。短按 长亮；请问那个团队修改最清晰，速度最快 代码维护更容易。

这个例子只是告诉大家，适合自己团队的才是最好的；虽然更细的颗粒度，扩展性，代码层级哪些都更好，但是也是和付出的人力相关的

1.6.3 指定对象接口

1 “指定对象接口”

- **意思：**给每个对象定义一组公开的操作或方法，这些就是对象的“接口”。
- **嵌入式 C 对应：**
 - 可以用函数 + 结构体封装（`struct + 函数指针`），或直接在头文件暴露 API
 - 例子：LED 对象提供 `led_on()`、`led_off()` 接口，外部代码调用这些接口，而不直接操作寄存器。

2 “暴露接口而隐藏实现”

- **意思：**对象内部的实现细节对外部不可见，调用者只关心接口功能。
- **嵌入式 C 对应：**
 - 头文件只声明接口函数
 - 驱动文件内部实现具体操作寄存器、定时器等
 - 优点：实现可以改动，调用者无需修改。

3 “接口的作用”

- **模块化：**每个对象内部逻辑可以随时改动，不影响外部调用。
- **解耦：**调用者只关心“做什么”，不关心“怎么做”。
- **可替换性：**同一个接口可以有多种实现（不同硬件或算法）。
- **嵌入式 C 对应：**
 - 通过 `struct + 函数指针` 或统一 API，可以替换不同硬件驱动，而不改调用代码。

4 与过程调用的对比

特点	直接过程调用	指定对象接口
可读性	简单	稍复杂但逻辑清晰
耦合	高	低

特点	直接过程调用	指定对象接口
扩展性	差	好
多硬件支持	麻烦	容易

- 在嵌入式 C 里，“指定对象接口”就是 **在有限资源下模拟面向对象设计**，让模块职责清晰、可替换、易维护。
- 当然，对于简单硬件、短生命周期项目，直接过程调用也完全可行。

1.6.4 描述对象的实现

1 原文大意拆解

1. 抽象类 (Abstract Class)

- 提供 **部分实现 + 接口声明**
- 调用者可以依赖抽象类接口，而不必关心具体实现

2. 混入类 (Mixin Class)

- 提供一些额外功能，可与其他类组合
- 不独立存在，只是为了扩展已有类的行为

3. 类继承 vs 接口继承

- **类继承**：继承父类的实现和接口
 - 调用者可能依赖父类实现细节 → 耦合度高
- **接口继承**：只继承方法签名（接口），不继承具体实现
 - 调用者只依赖接口 → 低耦合，可替换不同实现

4. 核心思想：

对接口编程，而不是对实现编程

- 调用者只依赖对象提供的接口，而不依赖具体实现或父类内部逻辑
- 便于模块替换、扩展和维护

2 嵌入式 C 的对应实现

C 语言没有类、继承和混入类，但可以模拟接口和抽象概念：

1. 抽象类对应 C

- 用 `struct + 函数指针` 定义接口和默认实现
- 外部模块依赖接口函数，不关心内部实现

```
typedef struct Motor Motor;

struct Motor {
    void (*start)(Motor* self);
    void (*stop)(Motor* self);
    // 默认实现可以提供空函数或基础功能
};
```

1. 接口继承对应 C

- 通过 **统一接口 struct**，不同模块实现同样方法
- 调用者只依赖接口，不管具体实现

```
typedef struct {
    void (*start)(void*);
    void (*stop)(void*);
} MotorOps;

typedef struct {
    int id;
    MotorOps* ops;
} Motor;
```

- 不同硬件可实现不同 `MotorOps`，调用者只用 `ops->start(&motor)`

1. 混入类对应 C

- 通过额外的函数或辅助 struct 实现额外功能
- 可组合到多个对象中使用

```
typedef struct {
    void (*log_status)(void*);
} LoggerMixin;

// Motor 对象可以组合 LoggerMixin，实现状态记录
```

1. 接口编程 vs 实现编程

- **接口编程**：调用 `ops->start(&motor)`，不依赖寄存器配置或算法
- **实现编程**：直接调用 `GPIOX->CR = 1` 等硬件寄存器操作 → 高耦合，修改成本大

3 总结

- **抽象类/接口** → 提供调用接口，隐藏实现
- **混入类** → 为对象增加可复用功能
- **接口继承**比类继承更灵活 → 调用者只依赖接口，耦合低
- **嵌入式 C 实现方式**:
 - `struct + 函数指针` 模拟对象接口

- 通过组合 struct 实现“混入”
- 调用者通过接口编程，而不是直接操作内部实现或寄存器

核心思想：**写代码依赖接口而不是实现** → 提高模块可替换性、扩展性和维护性。



设计模式

面向嵌入式软件开发



针对C语言及嵌入式软件开发的
伴读教程，陪伴学习设计模式这一
进阶知识点。

1.6.5 运用复用机制（嵌入式 C 精简伴读）

本节核心观点只有两个：

1. 软件复用≠复制代码，而是通过结构让代码可扩展、可替换、减少修改。
2. 复用方式中，“组合 + 委托”是最推荐的方案。

1. 什么是复用机制？

复用机制是指：

让新功能复用已有代码，不要强依赖内部细节，也不要到处修改老代码。

嵌入式 C 中常见复用手段：

- 组合（推荐）
- 伪继承（不推荐）
- 复制/修改（临时用）

2. 为什么推荐“组合”？

组合 = 结构体里放子模块 + 通过统一接口调用它们

在 C 中等价于：

- 模块内封装数据和行为
- 上层只通过函数指针表 / API 调用
- 新模块只换实现，不改旧代码

简单例子：统一通信接口

```
typedef struct {
    int (*send)(uint8_t*, int);
    int (*recv)(uint8_t*, int);
} io_ops_t;

typedef struct {
    io_ops_t* ops;    // 组合：放进去即可
} comm_t;
```

新增 UART / SPI / USB → 只需要新的 `ops`
上层协议无需修改。

3. 委托：组合的核心技巧

委托（Delegation）就是：

一个对象把工作交给另一个对象做。

在 C 中是：

- 主模块不直接实现行为
- 把行为“委托”给函数指针 / 回调

例子：

```
void protocol_send(comm_t* c, uint8_t* buf, int len) {
    c->ops->send(buf, len);    // 委托底层实现
}
```

上层不关心底层是 UART、SPI 还是虚拟链路。

4. 为什么不推荐“继承/伪继承”？

结构体嵌套模拟继承：

```
typedef struct { int id; } base_t;
typedef struct { base_t base; int extra; } child_t;
```

缺点：

- 耦合高
- 上层容易依赖内部结构
- 扩展困难

适合少量情况，不是主流复用手段。

1.6.6 关联运行时和编译时的结构（嵌入式 C 伴读）

本节原书讲的是：

类图（静态结构）和对象图（运行时结构）如何对应。

但在嵌入式 C 中：

没有类、没有对象生命周期管理、没有自动构造/析构机制。

因此这节的重点可以压缩为一句话：

C 里所有运行时关系都归结为“内嵌”或“指针引用”。

1. 编译时结构：就是你的 struct 定义

C 的“类图” = 结构体的定义方式。

示例：

```
typedef struct {  
    motor_t x;  
    motor_t y;  
} axis_t;
```

2. 运行时结构：你如何实例化 + 如何建立引用

对象图等价于：

- 哪些对象被创建
- 它们之间用什么指针链接

示例（运行时建立关系）：

```
motor_t motor_x, motor_y;  
  
axis_t axis = {  
    .x = motor_x,  
    .y = motor_y,  
};
```

运行时对象关系非常“物理”：

- 内嵌的就是内嵌
- 指针指向谁就是谁
- 没有隐藏行为

3. 聚合(Aggregation) vs 相识(Association) —— C 中基本无差别

在本节原书中，两个概念的区别是：

关系	意义
相识	A 只需要知道 B
聚合	A 拥有一个“弱成员” B（不负责生命周期）

但在嵌入式 C 中：

两者都是“指针引用”，语义上无法区分。

例子（相识 or 聚合？你看不出来）：

```
typedef struct {
    driver_t* drv;    // 指向外部，不负责分配
} protocol_t;
```

原书的“聚合”和“相识”概念，在 C 中都只是：

- 我知道你，但
- 你的生命周期不归我管
- 我只是一个指针

4. 嵌入式 C 中真正需要区分的只有两类关系

① 组合（Composition）= 内嵌结构体

```
typedef struct {
    sensor_t s;        // 我拥有你
} board_t;
```

特点：

- 生命周期绑定
- 内存布局固定（编译期确定）
- 强关系

② 引用（Reference）= 指针指向外部结构体

```
typedef struct {
    sensor_t* s;        // 我指向你，但不负责你
} board_t;
```

特点：

- 关系由运行时建立
- 生命周期不确定
- 弱关系（无法区分聚合 or 相识）

1.6.7 设计应支持变化（嵌入式 C 简明版）

这一小节的核心观点很简单：

设计要能承受未来变化，不要把变化写死，把稳定的部分抽出来、不稳定的部分隔离起来。

在 C 中，它的落地方式通常是：

- 用函数指针接口隔离变化
- 用组合 + 委托让行为可替换
- 把常变和不常变的东西拆开

1. 不要让变化渗入整个系统

嵌入式环境常见的“变化点”：

- 不同硬件驱动
- 不同通信链路（UART/SPI/USB）
- 不同协议版本
- 不同策略（校准算法、滤波算法）
- 不同业务流程（状态机）

错误做法是：

```
// 在上层写死底层细节
if (use_uart) {
    uart_send(...);
} else {
    spi_send(...);
}
```

这种写法导致：

底层一变，上层全改。

2. 正确做法：把变化部分抽象成接口（函数指针表）

例子：通信接口抽象

```
typedef struct {
    int (*send)(uint8_t*, int);
    int (*recv)(uint8_t*, int);
} io_ops_t;
```

上层不管是 UART、SPI、USB，只使用 `ops->send()`。

上层只依赖 **稳定的接口**，底层是变化的，实现可替换。

3. 使用组合 + 委托，把变化封装进去

```
typedef struct {
    io_ops_t* ops;        // 组合
} protocol_t;

int proto_send(protocol_t* p, uint8_t* buf, int len) {
    return p->ops->send(buf, len); // 委托给底层
}
```

未来要支持：

- SPI
- 两路 UART
- 无线链路

只要换 `ops`，上层完全不动。

这就是本节要表达的“支持变化”。

4. 关于“稳定 vs 不稳定部分”的拆分

嵌入式项目里的 **稳定部分**：

- 数据结构
- 公共接口
- 驱动抽象层
- 状态机框架
- 中断管理框架
- 日志、队列、协议框架

不稳定部分：

- 某个具体芯片驱动
- 某个协议细节
- 校准/滤波算法
- 项目特定业务流程

本节重点是：

把不稳定部分放在框架外侧，通过接口隔离开。

稳定部分形成平台 / 底座。



设计模式

面向嵌入式软件开发



针对C语言及嵌入式软件开发的
伴读教程，陪伴学习设计模式这一
进阶知识点。

1.7 怎样选择设计模式（嵌入式 C 视角）

1. 原则

1. 看变化点

- 哪些模块会经常改动？
- 哪些硬件/算法/协议可能换？

2. 看复用需求

- 是否希望相同框架支持多硬件/算法？
- 是否需要在不同项目中复用模块？

3. 看耦合度

- 上层是否依赖底层实现？
- 是否需要抽象接口来隔离变化？

2. 嵌入式 C 的对应做法

设计模式类型	C 里对应实现	适用场景
策略模式 (Strategy)	函数指针 + struct	不同算法可替换，如滤波器、PID
模板方法 (Template Method)	框架函数 + 回调	固定流程，可插入变化点
代理 / 委托 (Proxy / Delegation)	函数指针/ops委托底层实现	抽象硬件接口、延迟绑定模块
组合模式 (Composite)	struct 内嵌或组合	多模块组合，例如三轴平台控制
适配器模式 (Adapter)	包装函数/struct	不改上层逻辑，替换底层硬件接口

选择方式总结：

先找变化 → 看是否需要复用 → 决定用哪种模式隔离变化

目 的	设计模式	可变的方面
创建	Abstract Factory(3.1)	产品对象家族
	Builder(3.2)	如何创建一个组合对象
	Factory Method(3.3)	被实例化的子类
	Prototype(3.4)	被实例化的类
	Singleton(3.5)	一个类的唯一实例
结构	Adapter(4.1)	对象的接口
	Bridge(4.2)	对象的实现
	Composite(4.3)	一个对象的结构和组成
	Decorator(4.4)	对象的职责，不生成子类
	Facade(4.5)	一个子系统的接口
	Flyweight(4.6)	对象的存储开销
	Proxy(4.7)	如何访问一个对象；该对象的位置
行为	Chain of Responsibility(5.1)	满足一个请求的对象
	Command(5.2)	何时、怎样满足一个请求
	Interpreter(5.3)	一个语言的文法及解释
	Iterator(5.4)	如何遍历、访问一个聚合的各元素
	Mediator(5.5)	对象间怎样交互、和谁交互
	Memento(5.6)	一个对象中哪些私有信息存放在该对象之外，以及在什么时候进行存储
	Observer(5.7)	多个对象依赖于另外一个对象，而这些对象又如何保持一致
	State(5.8)	对象的状态
	Strategy(5.9)	算法
	Template Method(5.10)	算法中的某些步骤
	Visitor(5.11)	某些可作用于一个（组）对象上的操作，但不修改这些对象的类

1.8 怎样使用设计模式（嵌入式 C 视角）

1. 浏览模式 → 确认适用性

- 先看这个模式是否解决你的问题
- 仅在真的需要灵活性/复用时使用

// 例：传感器算法可能变化 → Strategy 模式适用

2. 理解结构 → 找参与者

- struct + 函数指针表 = 模式中的“类和对象”
- 组合 + 委托 = 模式中的“协作”

3. 看示例 → 找实现思路

- 参考 struct + ops + 委托实现
- 了解数据和函数指针如何交互

4. 映射到你的模块

- 给 struct/函数起业务相关名字
- 把模式角色映射到实际硬件/算法模块

```
filter_if_t kalmanFilter;    // Strategy 角色
sensor_t sensor = { .filter = &kalmanFilter };
```

5. 定义接口和数据结构

- 定义 struct
- 定义函数指针（接口）
- 组合可替换模块

```
typedef struct {
    filter_if_t* filter;
} sensor_t;
```

6. 定义操作（命名与职责）

- 上层调用统一接口
- 内部委托给具体实现

```
int sensor_read(sensor_t* s) {
    int v = read_hardware();
    return s->filter ? s->filter->apply(v) : v;
}
```

7. 实现操作 → 完成模式落地

- 填充函数指针
- 完成协作逻辑

```
int value = sensor_read(&sensor);
```

- 替换 filter → 上层不动

8. 使用注意事项

- 只在真正需要灵活性/复用时使用
 - 避免过度增加间接层，尤其是性能敏感的嵌入式模块
-

一句话总结流程：

确认模式 → 理解结构 → 映射模块 → 定义接口 → 委托实现 → 上层调用 → 替换模块时不改上层。



设计模式

面向嵌入式软件开发



针对C语言及嵌入式软件开发的
伴读教程，陪伴学习设计模式这一
进阶知识点。

1、意图

在系统初始化阶段，选择一个“平台实现”，之后系统中所有相关模块都必须从这个平台创建，且彼此匹配。

注意三点：

1. 一系列对象（不是一个）
2. 相关 / 依赖（必须能一起工作）
3. 选择发生在“创建阶段”，不是使用阶段

2、动机

2.1 原意

在 GoF 写书的年代（1995）：

GoF是“四人组”（Gang of Four）的英文缩写，指代软件工程著作《Design Patterns: Elements of Reusable Object-Oriented Software》的四位作者埃里希·伽玛（Erich Gamma）、理查德·海尔姆（Richard Helm）、拉尔夫·约翰逊（Ralph Johnson）和约翰·弗利赛迪斯（John Vlissides）。

- Windows / Motif / MacOS UI
- 控件长得不一样
- 但“按钮 / 滚动条 / 窗口”这些概念是一样的
- 用户代码希望 一次切换平台，整体换外观

GUI 是一个极佳的“产品族”例子：

windows 风格

- └ Button
- └ Scrollbar
- └ Menu

Motif 风格

- └ Button
- └ Scrollbar
- └ Menu

如何保证 Button / Scrollbar / Menu 来自同一个 UI 风格？

2.2 嵌入式适配

不是 RTOS，也不是驱动函数，而是：

平台 / 芯片 / BSP

因为它们同样具备三个特征：

- 1. 提供一组功能等价的接口
- 2. 实现细节强相关
- 3. 必须整体切换

GoF GUI	嵌入式等价
Button	UART
Scrollbar	GPIO
Menu	SPI
UI Toolkit	MCU / SoC / BSP

你觉得抽象工厂是怎么做的？stm32hal库？

Step 1：系统要支持多个平台

例如：

- STM32Fx
- GD32?

业务逻辑完全一样：

```
log_print("hello");
led_on();
```

Step 2: 所谓的实现?

```
#ifdef STM32Fx
    stm32_uart_send(...);
    stm32_gpio_set(...);
#elif defined GD32
    gd32_uart_send(...);
    gd32_gpio_set(...);
#endif
```

GoF 在 GUI 里用的是:

```
if (windows)
    new WinButton;
else
    new MotifButton;
```

Step 3: 问题暴露

1. 平台判断扩散到业务层

业务代码本应只关心“发日志”“点灯”，却被迫知道当前是 STM32 还是 GD32。
平台选择不再是架构决策，而是被拆散成大量 `#ifdef`。

2. 无法保证“产品族一致性”

编译层面允许:

- STM32 的 UART
 - GD32 的 GPIO
- 同时存在于同一系统中，但这种组合在语义上是错误的。

3. 平台切换成本极高

新增或替换一个平台:

- 需要修改大量业务文件
- 风险随代码规模线性上升

4. 创建逻辑与使用逻辑强耦合

业务代码在“使用外设”的同时，还承担了“选择外设实现”的职责，这正是 GoF 在 GUI 场景中明确反对的设计。

5. 系统初始化阶段缺乏统一的“平台绑定锚点”

平台选择是零散发生的，而不是在系统启动时一次性确定，导致系统整体架构不稳定、不可控。

3、适用性

1. 系统需要支持多个平台或芯片系列

- 同一套业务逻辑需要运行在不同 MCU / SoC 上
- 例如：STM32、GD32、NXP、国产替代芯片
- 平台差异主要集中在外设实现层

典型特征：

平台可切换，但业务不可改。

2. 系统中存在“必须成组使用”的外设或模块

- UART / GPIO / SPI / I2C 等外设之间存在隐性耦合
- 时钟树、DMA、中断模型通常是平台级统一设计
- 混用不同 BSP 的外设实现在语义上是错误的

抽象工厂的价值：

保证外设对象始终来自同一平台族。

3. 平台选择发生在系统初始化阶段

- 平台在上电或启动阶段即可确定
- 运行期不需要、也不允许动态切换平台
- 非运行期策略选择，而是系统架构决策

这与 GoF“创建阶段选择实现”的假设完全一致。

4. 业务代码需要与平台实现彻底解耦

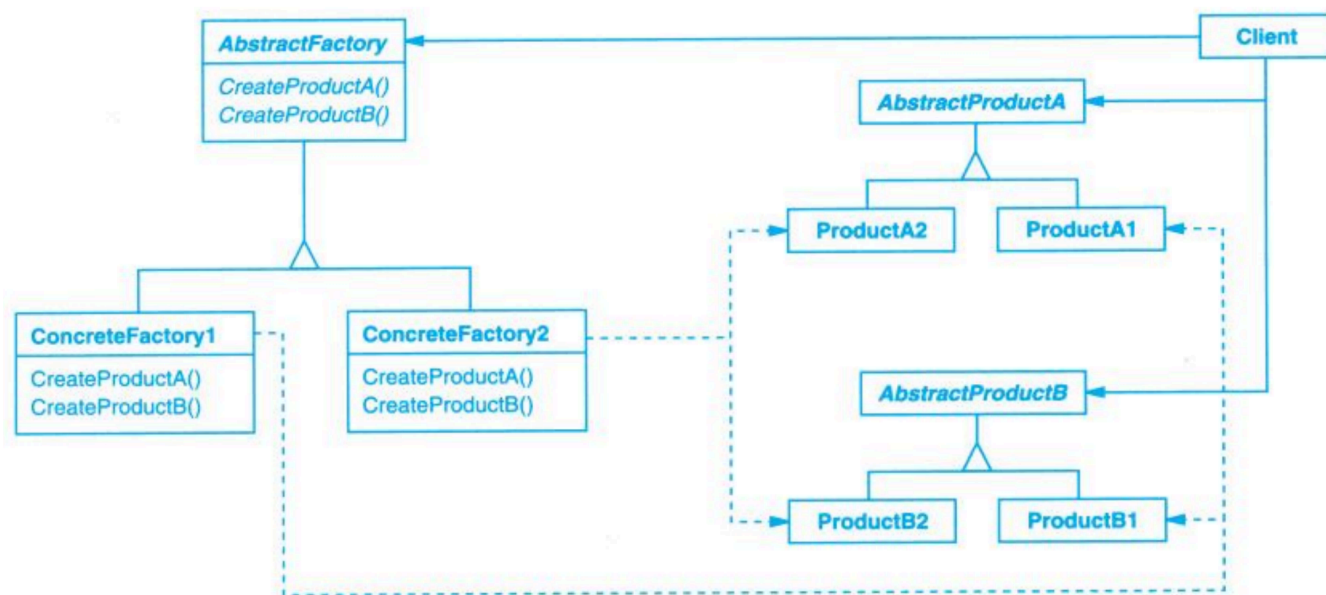
- 业务逻辑只关心“能力”，不关心“实现”
 - 禁止在业务层出现 `#ifdef MCU_xxx`
 - 便于代码复用、单元测试、仿真与移植
-

5. 不适用的情况（嵌入式中常见误用）

- 仅支持单一平台，且长期不考虑移植
- 外设数量极少，平台差异可以忽略
- 对代码体量、抽象层次极度敏感的极小系统

**此时引入抽象工厂只会增加复杂度。

4、结构



Client 需要能力

Client 只知道 AbstractFactory

AbstractFactory 声明能创建哪些 AbstractProduct

ConcreteFactory 决定用哪套 ConcreteProduct

ConcreteProduct 实现真正的平台细节

5、参与者（Participants）——嵌入式视角

1. AbstractFactory（抽象工厂）

角色含义：

定义平台能够创建的一组外设或系统能力的接口。

嵌入式等价：

PlatformFactory / BSPFactory

职责：

- 声明可创建的外设能力（UART / GPIO / SPI 等）
- 不包含任何平台或芯片相关信息
- 作为业务层唯一可见的“平台入口”

2. ConcreteFactory（具体工厂）

角色含义：

实现 AbstractFactory，绑定到某一个具体平台。

嵌入式等价：

- STM32Factory
- GD32Factory
- NXPFactory

职责：

- 在系统初始化阶段被选中
 - 负责创建该平台下的所有外设实现
 - 保证生成的外设对象来自同一 BSP / 芯片族
-

3. AbstractProduct（抽象产品）

角色含义：

定义某一类对象的公共接口。

嵌入式等价：

- UART
- GPIO
- SPI

职责：

- 描述“能力”，而不是“实现”
 - 不依赖寄存器、HAL 或具体外设型号
 - 作为业务层操作外设的唯一接口
-

4. ConcreteProduct（具体产品）

角色含义：

实现 AbstractProduct，对应具体平台下的实现。

嵌入式等价：

- STM32_UART
- GD32_UART
- STM32_GPIO

职责：

- 封装寄存器访问、HAL 调用、中断与 DMA 细节
- 仅由对应的 ConcreteFactory 创建

- 不应被业务代码直接引用
-

5. Client（客户端）

角色含义：

使用抽象工厂和抽象产品的代码。

嵌入式等价：

业务逻辑 / 应用层 / 控制逻辑

职责：

- 仅依赖 AbstractFactory 和 AbstractProduct
 - 不关心具体平台实现
 - 不参与平台选择，也不参与外设创建策略
-

6、协作（Collaborations）——嵌入式视角

1. 系统启动阶段选择 ConcreteFactory

- 根据编译选项或启动配置确定具体平台
- 该选择只发生一次

2. Client 持有 AbstractFactory 接口

- 业务代码仅知道“平台工厂”，不知道具体平台
- 不直接创建任何外设对象

3. AbstractFactory 负责创建 AbstractProduct

- Client 通过工厂接口获取 UART / GPIO / SPI 等能力
- 不关心具体实现来源

4. ConcreteFactory 实例化 ConcreteProduct

- 每个 ConcreteFactory 只创建自身平台对应的外设实现
- 保证外设对象来自同一 BSP / 芯片族

5. Client 通过 AbstractProduct 使用外设能力

- 业务代码只调用抽象接口
- 不参与平台判断，也不感知平台差异

7、效果（Consequences）——嵌入式视角

- 强制平台一致性
外设对象成族创建，避免混用不同 BSP / 芯片实现。
- 业务层与平台解耦
业务代码只依赖能力接口，不再感知 MCU 或 HAL。
- 平台切换成本可控
新增或替换平台，修改范围集中在工厂实现。

- **结构清晰，架构边界明确**
平台选择成为初始化阶段的显式决策，而非隐性 `#ifdef`。

- **抽象层增加**
代码量与接口数量上升，不适合极小系统。
- **平台扩展需要成体系实现**
新平台必须完整实现一组外设能力，而不能零散接入。
- **运行期灵活性有限**
设计目标是“初始化期绑定”，不支持运行期平台切换。

8、已知应用

平台类别	具体品牌/库	平台工厂 (AbstractFactory / ConcreteFactory)	外设能力 (AbstractProduct)	具体实现 (ConcreteProduct)
MCU / 单片机	STM32 / GD32 / NXP / TI	BSPFactory / PlatformFactory → STM32Factory / GD32Factory / iMXRTFactory / TivaFactory	UART / GPIO / SPI / I2C / PWM	HAL / DriverLib 等 MCU 具体驱动
Linux / SoC	x86 / ARM / RISC-V	PlatformFactory → x86Factory / ARMFactory	GPIO / UART / SPI / I2C	libgpiod / i2c-dev / spidev
仿真 / 测试平台	QEMU / HIL 仿真板	PlatformFactory → QEMUFactory / SimBoardFactory	UART / GPIO / SPI / ADC	虚拟外设对象 / 仿真驱动

9、相关模式示例

- **Factory Method**
 - 抽象工厂内部每个外设对象由工厂方法创建
- **Bridge**
 - 分离接口能力和具体实现，外设能力独立于芯片
- **Singleton**
 - 平台工厂通常全局唯一，初始化一次即可



设计模式

面向嵌入式软件开发



针对C语言及嵌入式软件开发的
伴读教程，陪伴学习设计模式这一
进阶知识点。

补充讲义：抽象工厂在嵌入式驱动层的局限性

1、之前讲义的例子回顾

- 描述了抽象工厂用于 MCU 平台选择（STM32/GD32/NXP）
- 假设业务代码只依赖抽象工厂创建的 UART/GPIO/SPI 对象
- Client 完全不感知具体平台，实现平台无关业务逻辑

理论上，这符合 GoF 抽象工厂定义：一次选择，创建一组成套对象族。

2、实际嵌入式工程问题

2.1 驱动层不需要抽象工厂

假设	实际情况
每个外设（UART/GPIO/SPI）由工厂创建	MCU HAL 或 RTOS 设备框架本身提供统一接口 + 函数表 + 注册机制
业务逻辑不关心具体平台	HAL/RTOS 驱动接口已解耦寄存器和具体芯片型号，业务直接调用即可
平台切换通过 ConcreteFactory	实际 MCU 工程通常通过编译选项、BSP 选择或宏定义实现；运行期动态切换不常见

结论：对底层驱动和外设对象使用抽象工厂会导致**结构复杂、额外抽象层、运行期开销增加**，而工程上没有实质收益。

2.2 抽象工厂与 MCU HAL / RTOS 设备框架对比

特性	抽象工厂设计	HAL / RTOS / Linux 驱动模型
对象创建	ConcreteFactory 负责实例化成套 ConcreteProduct	驱动注册/设备表 + ops 表，业务直接调用接口
平台切换	初始化期选择 ConcreteFactory	编译期 BSP 或 Makefile 选择，运行期一般固定
可维护性	增加工厂/抽象产品数量，代码量大	驱动接口统一，易扩展且贴合 MCU 架构
灵活性	可动态替换对象族，但嵌入式不常用	支持动态绑定驱动和设备（Linux/RTOS）
适用场景	多 SKU / 多平台产品族，方案初始化期绑定	单平台 MCU 或 HAL 层抽象，外设调用一致接口即可

3、现实做法建议

- 1. 驱动层/底层外设
 - 统一接口 + 函数表 + 注册机制
 - MCU HAL / LL / RTOS device model
 - 避免为每个 UART/GPIO 创建工厂
- 2. 算法/策略可替换
 - 使用 Strategy 模式
 - 适合运行期选择控制算法或业务策略
- 3. 产品线/方案族切换
 - 适合抽象工厂
 - 初始化期选择 ConcreteFactory，创建一整套配套模块
 - 典型例子：智能家居不同 SKU（Basic/Pro/Industrial）组合显示、通信、执行器

4、总结

- 之前讲义例子的问题：把抽象工厂用于每个 MCU 外设创建，实际嵌入式工程不需要，也不符合 HAL/RTOS 实际模式。
- 核心理念依然有价值：保证“同一方案/平台下对象族一致性”，适合产品族层面。
- 底层驱动抽象不适合工厂：统一接口 + 注册/ops 表更贴近工程实践。



设计模式

面向嵌入式软件开发



针对C语言及嵌入式软件开发的
伴读教程，陪伴学习设计模式这一
进阶知识点。

Builder 模式讲义

—— 以“多层嵌套协议构造”为例（嵌入式语境）

1 Intent（意图）

将一个复杂对象的构建与其表示分离，使得同样的构建过程可以创建不同的表示。

在嵌入式协议场景中：

- 复杂对象 = 一帧多层嵌套协议报文
- 构建过程 = 按顺序拼装各层结构
- 表示 = 不同版本/不同安全级别/不同扩展字段的协议格式

核心理解：

把“如何构造报文”的算法独立出来，而不是让调用者手工拼 buffer。

2 Motivation（动机）

考虑一个嵌入式常见的多层协议帧结构：

```
Frame
├── HeadInfo
├── ExtHeader (optional)
├── SecurityBlock (optional)
├── Payload
│   ├── SubHeader
│   ├── TLV1
│   └── TLV2
└── CRC
```

在实际系统中，构造顺序通常是：

1. 先组装 Payload（TLV / 数据块）
2. 根据 Payload 长度生成 SubHeader
3. 如启用安全功能，进行加密或签名
4. 再填写 HeadInfo（总长度、版本、标志位等）
5. 最后计算 CRC

特点：

- 构造顺序与最终字节布局顺序不同
- HeadInfo 中的长度字段依赖 Payload
- SecurityBlock 依赖完整数据
- CRC 依赖整个 Frame
- 不同版本可能增加 ExtHeader 或 SecurityBlock

如果由调用者自行拼接：

- 容易遗漏长度回填
- 容易错误计算校验范围
- 容易破坏字段依赖关系
- 版本扩展会导致大量 if/else 分支

问题本质：

报文构造存在严格的步骤顺序和字段依赖关系，
同时协议结构会随着版本演进而变化。

3 Applicability（适用性）

在嵌入式多层协议中，当满足以下条件时适合使用 Builder：

✔ 对象复杂

- 多层嵌套
- 多字段依赖
- TLV 可变数量

✔ 构造顺序严格

- 必须先构造 Payload
- 再计算签名
- 再计算 CRC

✔ 同一构造流程支持不同表示

例如：

- 普通帧
- 加密帧
- 带扩展字段帧
- Debug 帧

构造步骤类似，但内部结构不同。

✔ 希望隔离表示细节

上层模块只知道：

“构造一个合法协议帧”

而不关心字段细节。

4 Structure（结构）

在协议场景中的角色映射：

GoF 角色	协议场景对应
Builder	报文构造接口
ConcreteBuilder	不同协议版本/安全级别实现
Director	报文构造流程控制
Product	最终字节流 buffer

结构逻辑：

```
Application
  ↓
Director (构建流程)
  ↓
Builder (抽象接口)
  ↓
ConcreteBuilder (具体协议格式)
  ↓
Buffer (最终报文)
```

5 Participants (参与者)

1. Builder

定义构建步骤接口，例如：

- build_header()
- build_security()
- build_payload()
- build_crc()

注意：

这里的关键不是函数名，而是：

步骤被标准化。

2. ConcreteBuilder

实现不同版本：

- Version1Builder
- SecureBuilder
- ExtendedBuilder

不同实现：

- 是否包含 Security Block
- TLV 顺序是否不同
- CRC 算法是否不同

3. Director

负责构建顺序，例如：

1. 构造 Payload
2. 构造 Header

3. 插入 Security
4. 计算 CRC
5. 输出 Frame

关键点：

Director 不关心具体字段结构。

4. Product

最终报文缓冲区。

它不暴露内部拼接细节。

6 Collaborations（协作）

构建过程：

1. Client 创建 ConcreteBuilder
2. Director 按顺序调用构建步骤
3. Builder 内部维护中间状态
4. 构建完成后返回完整 Frame

关键协作特征：

- Client 不直接操作 Buffer
 - Director 控制顺序
 - Builder 负责细节
-

7 Consequences（效果）

结合嵌入式协议场景分析。

① 分离构造算法与表示

构造算法固定：

- 先构造 payload
- 再插入 header
- 再计算校验

表示变化：

- 是否加密
- 是否签名
- TLV 顺序

好处：

协议版本升级时，不需要修改构造流程。

② 更好的结构控制

防止：

- CRC 计算范围错误
- 长度更新错误
- 可选字段遗漏

Builder 将不变量集中管理。

③ 易于扩展新协议版本

新增：

- 新安全机制
- 新扩展字段

只需新增 ConcreteBuilder。

④ 隔离复杂性

应用层代码变为：

“构造一帧合法报文”

而不是：

“逐字节拼接”

8 Implementation（实现要点）

在嵌入式 C 中实现时应注意：

① 内部维护中间状态

例如：

- 当前写入位置
- 当前 payload 长度
- 是否已构造安全块

Builder 必须维护这些。

② 防止非法构造顺序

可以：

- 在内部维护状态机
- 防止未完成 payload 就计算 CRC

③ 避免动态内存（嵌入式约束）

通常：

- 使用固定 buffer
- 预分配空间
- 构造完成后返回长度

④ Director 可以内嵌

在嵌入式中：

- Director 常为一个封装函数
- 不一定单独定义模块

9 与其他模式对比（协议场景）

模式	在协议构造中的角色
Factory	选择创建哪种 Builder
Strategy	选择 CRC 算法
Template Method	固定流程，但不隔离表示
Builder	隔离构造算法与表示

在多层嵌套协议中：

Builder 往往和 Strategy 组合使用。

10 总结（嵌入式视角）

在嵌入式多层嵌套协议中：

Builder 解决的问题不是：

“如何创建对象”

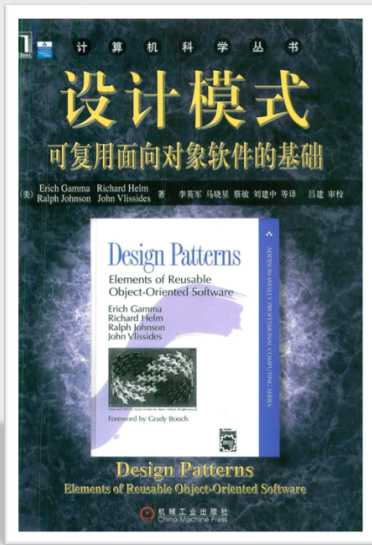
而是：

“如何保证复杂报文按正确顺序、正确依赖关系被构造”

当协议具有：

- 嵌套结构
- 版本差异
- 校验依赖
- 可选字段
- 安全模块

Builder 是一种合理且工程上可落地的组织方式。



设计模式

面向嵌入式软件开发



针对C语言及嵌入式软件开发的
伴读教程，陪伴学习设计模式这一
进阶知识点。

工厂方法（Factory Method）

一、核心定义

定义一个创建对象的接口，但将具体对象的创建延迟到子模块（具体驱动）中完成。

在嵌入式 C 语言中通常表现为：

- 框架规定创建流程
- 具体驱动完成实例构造
- 通过函数指针实现多态

二、结构抽象（通用模型）

框架层（Creator）

```
|  
|-- 调用创建接口  
|  
--> 具体驱动实现创建逻辑  
|  
--> 构造具体设备对象  
--> 绑定操作函数表
```

关键点：

- 框架不直接实例化具体对象

- 创建逻辑由具体实现提供
- 使用与创建分离

三、典型场景（嵌入式）

适合使用场景：

- 驱动框架
- 设备模型
- 总线匹配机制
- 协议栈注册
- 文件系统挂载

不适合：

- 单一固定硬件
- 无扩展需求的小系统

四、案例一：RT-Thread 设备框架

1 抽象设备结构

```
struct rt_device
{
    const struct rt_device_ops *ops;
    void *user_data;
};
```

`ops` = 操作接口（多态）

2 具体驱动实现操作表

```
static const struct rt_device_ops uart_ops =
{
    uart_init,
    uart_open,
    uart_close,
    uart_read,
    uart_write,
};
```

3 驱动初始化函数（工厂方法）

```
int rt_hw_uart_init(void)
{
    static struct rt_device uart_dev;

    uart_dev.ops = &uart_ops;

    rt_device_register(&uart_dev, "uart1", RT_DEVICE_FLAG_RDWR);

    return 0;
}
```

分析

- 驱动负责构建设备对象
- 框架只提供注册入口
- 上层通过统一接口访问

这就是 C 语言环境下的工厂方法。

五、案例二：Linux 驱动 probe 机制

1 抽象接口

```
struct file_operations {
    ssize_t (*read)(...);
    ssize_t (*write)(...);
};
```

2 具体驱动实现

```
static const struct file_operations my_fops = {
    .read = my_read,
    .write = my_write,
};
```

3 probe 函数（工厂方法）

```
static int my_driver_probe(struct platform_device *pdev)
{
    struct my_dev *dev;

    dev = devm_kzalloc(&pdev->dev, sizeof(*dev), GFP_KERNEL);

    dev->fops = &my_fops;

    platform_set_drvdata(pdev, dev);

    return 0;
}
```

分析

- 总线匹配成功后调用 probe
- 驱动在 probe 中创建具体设备实例
- 绑定操作函数表
- 框架不关心具体实现类型

六、模式特征总结

在嵌入式 C 中，工厂方法通常具备：

- 初始化函数 / probe 函数
- 注册机制
- 函数指针表实现多态
- 框架与驱动解耦

表现形式不是：

- 继承
- virtual 函数

而是：

结构体 + 函数指针 + 注册机制

七、优点

- ✓ 解耦框架与驱动
- ✓ 易于扩展新设备
- ✓ 支持模块化加载（Linux）
- ✓ 符合分层架构设计

八、代价

- ✗ 引入函数指针复杂度
- ✗ 调试路径更长
- ✗ 代码理解成本增加

在资源极小 MCU 上需要权衡。

九、总结

在嵌入式系统中：

工厂方法并不是“类继承 + virtual”的形式，
而是通过“驱动初始化 / probe + 函数指针表”实现对象创建的延迟与解耦。

RT-Thread 设备模型和 Linux 驱动模型是成熟的工业级实现。

补充：RT-Thread 与 Linux 是否属于工厂方法？

在嵌入式实际工程中，工厂方法通常不会以“继承 + virtual”的形式出现，而是以**驱动注册机制**或 **probe 机制**实现。

1 Linux 驱动模型 —— 典型工厂方法

核心流程：

```
设备匹配成功
    ↓
调用 driver->probe()
    ↓
probe 创建具体设备实例
    ↓
绑定操作函数表
```

特点：

- 框架定义创建时机
- 具体驱动实现创建逻辑（probe）
- 框架不直接实例化具体设备
- 创建与使用分离

结论：

Linux driver model 是工程化实现的典型工厂方法。

2 RT-Thread 设备框架 —— 注册式工厂方法

核心流程：

```
驱动初始化
  ↓
构建设备对象
  ↓
绑定 ops 表
  ↓
rt_device_register()
```

特点：

- 设备对象由具体驱动构造
- 框架只负责统一管理和调用
- 上层通过抽象接口访问

与 Linux 不同之处：

- Linux 是“框架调用子类创建”
- RT-Thread 是“驱动主动注册”

但本质相同：

创建逻辑由具体实现决定，而非框架统一实例化。

结论：

RT-Thread 属于注册式工厂方法变体。



设计模式

面向嵌入式软件开发



针对C语言及嵌入式软件开发的
伴读教程，陪伴学习设计模式这一
进阶知识点。

Prototype 模式讲义（简化版）

1 Intent（意图）

通过复制已存在的原型实例来创建新对象，而不是每次手动初始化。

2 核心思想

在嵌入式 C 中：

- **Prototype**：一个已有对象作为“模板”
- **复制**：通过复制模板来创建新对象
- **区别**：不需要重复手动初始化每个字段

3 何时使用

适用于：

- 对象结构复杂，需要初始化的字段多
- 多个对象实例相似，差异少
- 默认配置多次使用，想集中管理

不适用于：

- 对象初始化非常简单

- 需要依赖硬件配置的对象

4 Structure（结构）

- **Prototype**：模板实例（const 结构体）
- **Client**：通过模板创建新对象
- **clone()**：复制操作

创建过程：

1. 客户端获取原型模板
2. 执行复制
3. 修改少量差异字段

5 优点与缺点

优点

- 统一管理默认配置
- 减少初始化代码
- 支持动态扩展（新增模板即可）

缺点

- 深拷贝风险（需要处理指针成员）
- 可能误修改原型

6 实现要点

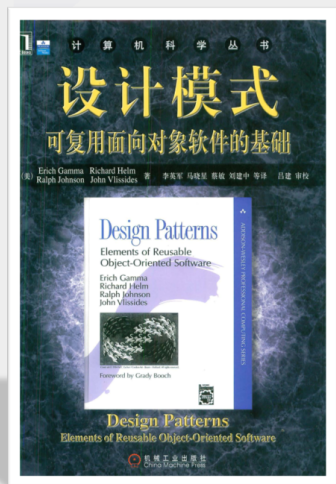
- 模板实例为只读（防止修改）
- 复制时注意深拷贝（指针成员）
- 适当结合内存池管理对象

7 总结

在嵌入式 C 中，Prototype 模式通过：

复制已有的模板对象来创建新对象，避免每次重复初始化。

它让对象创建更加简洁、统一、可扩展。



设计模式

面向嵌入式软件开发



针对C语言及嵌入式软件开发的
伴读教程，陪伴学习设计模式这一
进阶知识点。

单例模式（Embedded-Oriented Singleton）

1. 意图（Intent）

在系统中保证某类资源：

- 全局唯一实例
- 可控访问路径
- 受约束的生命周期
- 避免重复初始化

在嵌入式场景中，本质目标通常不是“对象唯一”，而是：

保证物理资源或内核核心控制结构的唯一性与一致性。

2. 动机（Motivation）

嵌入式系统具有天然的“单实例资源”特征：

- MCU 外设物理唯一（UART1、SPI2 等）
- 系统 Tick 唯一
- 调度器控制块唯一
- 当前任务指针唯一
- 内核状态机唯一

如果不进行结构约束：

- 可能重复初始化硬件

- 状态被多个模块无序修改
- 系统进入不可预测状态

典型问题示例

```
void uart_init(void);  
void uart_send(const char *data);
```

如果多个模块随意调用 `uart_init()`：

- 可能重复配置寄存器
- 打断正在进行的 DMA
- 破坏通信状态

嵌入式单例的核心目的：

把“资源唯一性”提升为架构约束。

3. 适用性 (Applicability)

在嵌入式系统中，以下场景适合单例：

3.1 架构级资源

- 调度器核心控制块
- 系统时间源
- 中断管理器
- 内核状态机

3.2 外设抽象层

- UART 驱动控制块
- SPI 总线管理器
- DMA 控制器
- Flash 驱动

3.3 协议栈核心

- TCP 活跃 PCB 链表
- 网络接口链表头
- BLE 协议栈全局状态

4. 结构 (Structure)

在 C 语言中没有 `class` 和 `private` 构造函数，因此结构通常表现为：

4.1 文件级静态实例

```
static struct uart_driver g_uart;
```

4.2 对外暴露访问接口

```
struct uart_driver* uart_get_instance(void)
{
    return &g_uart;
}
```

4.3 内部初始化控制

```
static int initialized = 0;

void uart_init(void)
{
    if (initialized)
        return;

    /* hardware configuration */
    initialized = 1;
}
```

5. 参与者 (Participants)

在嵌入式单例中，典型参与角色如下：

角色	描述
Singleton Instance	静态全局结构体
Access Interface	获取实例或访问 API
Client Modules	使用该资源的其他模块
Synchronization Mechanism (同步机制)	中断屏蔽 / 自旋锁 / 原子操作

6. 协作 (Collaborations)

6.1 单核 MCU

- 通过关中断保证一致性
- 无真正并行线程竞争

6.2 RTOS

- 调度器控制块在临界区更新
- 使用任务级同步机制

6.3 SMP 系统

- 使用 per-core 单例拆分竞争
- 或使用 spinlock 保护

7. 结果 (Consequences)

优点

1. 明确资源所有权
2. 防止重复初始化
3. 简化资源访问路径
4. 降低系统状态不一致风险
5. 减少动态内存使用

缺点

1. 强耦合
2. 难以单元测试
3. 难以做多实例仿真
4. 全局状态可能导致隐式依赖

嵌入式特有风险

- 中断与任务访问竞争
- SMP 下缓存一致性问题
- 初始化顺序依赖

8. 实现 (Implementation)

在嵌入式 C 中，常见实现方式有三类：

8.1 直接全局变量 (Linux 风格)

```
struct kernel_control_block kernel;
```

优点：简单直接

缺点：无封装

8.2 static + getter（模块化风格）

```
static struct scheduler sched;

struct scheduler* scheduler_instance(void)
{
    return &sched;
}
```

优点：

- 限制外部直接访问
- 可隐藏内部结构

8.3 每核心单例（SMP 优化）

```
DEFINE_PER_CPU(struct rq, runqueues);
```

特征：

- 每 CPU 唯一
- 降低锁竞争
- 提高可扩展性

8.4 并发控制策略

执行模型	控制方式
单核 + 中断	关中断
RTOS	临界区
SMP	spinlock
只读单例	不需要锁



设计模式

面向嵌入式软件开发



针对C语言及嵌入式软件开发的
伴读教程，陪伴学习设计模式这一
进阶知识点。

4.1 Adapter（适配器）

一、意图（Intent）

将一个类的接口转换成客户希望的另外一个接口。

Adapter 使得原本由于接口不兼容而不能一起工作的类可以一起工作。

在嵌入式领域，这通常表现为：

- 上层业务依赖某固定 API（如 FreeRTOS API）
- 但底层环境（如 PC 上的 POSIX）接口不同
- 通过适配器，将 POSIX 接口转换为 FreeRTOS 接口

二、又称（Also Known As）

Wrapper（包装器）

三、动机（Motivation）

假设：

- 应用代码基于 FreeRTOS：

```
xTaskCreate(task, ...);
```

- 现在希望在 PC 上运行该代码
- PC 环境只有 `pthread_create`

问题：

- `pthread_create` 接口形式不同

- 业务代码已经大量依赖 FreeRTOS API
- 不希望修改应用层

解决方案：

- 写一个适配层
- 在 PC 环境下，用 `pthread_create` 实现 `xTaskCreate`

于是：



这样无需修改应用层即可在 PC 运行。

四、适用性 (Applicability)

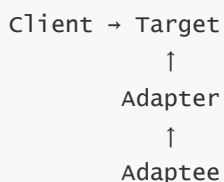
以下情况适合使用 Adapter：

1. 需要使用一个已存在的类，而其接口不符合你的需求
(例如 pthread 与 FreeRTOS API 不一致)
2. 希望创建一个可复用的类，与不相关或不可预见接口协同工作
3. 在嵌入式系统中：
 - RTOS 切换
 - HAL 统一
 - 老代码迁移
 - PC 仿真运行嵌入式逻辑

五、结构 (Structure)

在 C 语言中通常采用“对象适配器”方式（通过组合实现）。

结构关系：



在本例中：

- Client：应用任务代码
- Target：FreeRTOS API (`xTaskCreate`)
- Adaptee：`pthread_create`
- Adapter：`freertos_compat_posix.c`

六、参与者 (Participants)

1. Target

定义客户使用的特定领域接口。

在嵌入式场景中：

FreeRTOS 的任务 API

2. Client

与 Target 接口协作的对象。

例如：

业务任务代码

3. Adaptee

已经存在的接口，需要被适配。

例如：

POSIX pthread 接口

4. Adapter

将 Adaptee 的接口转换为 Target 接口。

在本例中：

- `xTaskCreate()` 内部调用 `pthread_create()`
 - 完成参数转换和封装
-

七、协作 (Collaborations)

Client 通过 Target 接口调用 Adapter。

Adapter：

1. 接收 Target 请求
2. 转换为 Adaptee 请求
3. 调用 Adaptee
4. 返回结果给 Client

在嵌入式示例中：

- 应用调用 `xTaskCreate`
- Adapter 将其转为 `pthread_create`
- pthread 创建线程
- 返回结果

八、效果 (Consequences)

优点

1. 将接口转换与业务逻辑分离
2. 提高可复用性
3. 减少对既有代码的修改
4. 支持平台迁移和环境仿真

缺点

1. 增加一层间接性
2. 可能无法完全匹配语义差异
(例如调度优先级、栈管理等)

在嵌入式中常见问题：

- FreeRTOS 的实时调度语义无法完全由 pthread 等价实现
- 适配器只能解决“接口不兼容”，不能保证“行为完全一致”

九、实现 (Implementation)

在 C 语言中没有继承机制，因此通常采用：

- 函数封装
- 结构体 + 函数指针
- 模块级封装

适配器实现关键点：

1. 不修改 Client
2. 不修改 Adaptee
3. 只在 Adapter 内完成接口转换

十、示例代码 (简要)

目标接口 (Target)：

```
BaseType_t xTaskCreate(TaskFunction_t task, ...);
```

适配器内部实现：

```
BaseType_t xTaskCreate(...)
{
    return pthread_create(...);
}
```

Client 无需修改。

十一、相关模式 (Related Patterns)

- **Bridge**
Bridge 旨在分离抽象与实现，使两者可以独立变化。
Adapter 用于事后接口兼容。
 - **Decorator**
Decorator 在不改变接口的情况下增加职责；
Adapter 改变接口。
 - **Facade**
Facade 提供统一接口，但不一定解决接口不兼容问题。
-

嵌入式场景总结

在嵌入式领域，Adapter 常见于：

- RTOS API 兼容层
- HAL 驱动统一
- 第三方库封装
- PC 仿真环境

核心思想不变：

当接口已经确定且不可修改时，通过一层适配器，使不兼容接口协同工作。



设计模式

面向嵌入式软件开发



针对C语言及嵌入式软件开发的
伴读教程，陪伴学习设计模式这一
进阶知识点。

一、Bridge 的核心意图（Intent）

将抽象部分与实现部分分离，使它们可以独立变化。

注意三个关键词：

- 抽象部分
- 实现部分
- 独立变化

Bridge 不是“抽象硬件”，
而是解决“两个维度同时变化”的问题。

二、嵌入式场景中的问题（Motivation）

以通信系统为例。

在一个嵌入式产品中，通常存在：

维度 A：协议类型（逻辑层）

- Modbus
- Bootloader 协议
- 私有业务协议
- CANopen

维度 B：传输方式（物理层）

- UART
- TCP
- CAN
- USB

这两个维度是正交的。

如果不使用 Bridge

我们往往会写成：

- Modbus over UART
- Modbus over TCP
- Bootloader over UART
- Bootloader over USB

结构上变成：

协议 × 传输方式

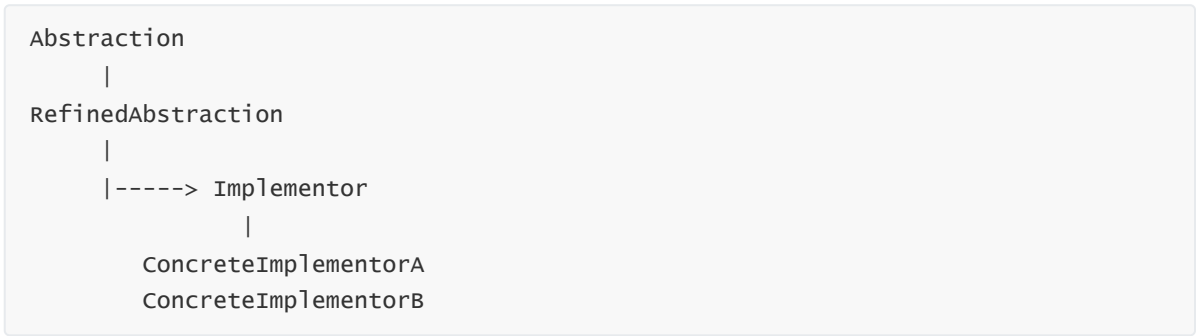
当协议数量为 M，传输方式为 N：

组合复杂度 = $M \times N$

这就是 GoF 书中所谓的“类爆炸问题”。

三、Bridge 的结构思想（Structure）

GoF 结构：



映射到嵌入式通信：



关键点：

- 协议层持有“传输接口”
- 协议不依赖具体 UART / TCP
- 传输层也不依赖具体协议

四、Bridge 在这里解决了什么？

1 把两个变化维度拆开

原本耦合在一起的：

协议 + 传输方式

被拆为：

- 协议层次结构
- 传输层次结构

两个层次结构通过“桥”连接。

2 从 $M \times N$ 变成 $M+N$

新增一个协议：

- 不改任何传输层代码

新增一个传输方式：

- 不改任何协议代码

这才是 Bridge 的核心价值。

五、Bridge 的本质理解（嵌入式角度）

在嵌入式中，Bridge 本质是：

业务逻辑层 与 硬件实现层 的结构性解耦
且双方都可能扩展

它不同于：

- HAL（单一维度抽象）

- Strategy（单一算法替换）

Bridge 一定是**两个维度**。

六、与工厂方法的关系

如果你前面已经讲过工厂方法，可以这样衔接：

- Factory 解决：创建哪个传输实现？
- Bridge 解决：协议如何与传输解耦？

一句话概括：

Factory 决定“用谁”
Bridge 决定“怎么连”

七、Bridge 在嵌入式中的典型应用类别

除了通信协议，还有：

场景	两个变化维度
文件系统	文件 API × 存储介质
GUI 系统	绘图逻辑 × 屏幕驱动
网络协议栈	协议 × 网卡
设备框架	设备抽象 × 具体芯片

但通信例子最清晰。

八、什么时候适合使用 Bridge？

当满足三个条件时：

1. 存在两个独立变化维度
2. 可能形成组合爆炸
3. 两个维度都需要独立扩展

如果只是单一抽象接口：

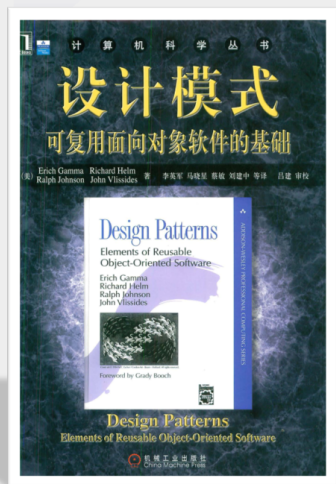
那更可能是 Strategy 或简单多态。

九、总结

在嵌入式系统中，当我们发现
协议种类在增加，
传输方式也在增加，
且两者相互独立时，

就应该考虑桥接模式。

它的目的不是“抽象硬件”，
而是控制系统结构复杂度。



设计模式

面向嵌入式软件开发



针对C语言及嵌入式软件开发的
伴读教程，陪伴学习设计模式这一
进阶知识点。

1. 意图 (Intent)

将对象组织成树形结构表示“部分-整体”层次，使客户端能够以一致的方式处理单个对象与组合对象。

在 FatFs 场景中：

- 文件是“部分”
- 目录是“整体”
- 整体可以包含部分，也可以包含新的整体
- 上层系统不应关心“它是文件还是目录”

组合模式关注的不是“文件读写”，而是：

如何表达一个递归的层级结构，并让操作在结构中自然传播。

2. 问题的抽象层次

先脱离 FatFs。

文件系统本质上是什么？

它不是 API 集合。

它是一个递归定义的层级结构：

一个目录由若干节点组成，每个节点要么是文件，要么是目录。

这是一个数学上的递归定义。

问题在于：

- 文件和目录语义不同
- 但它们在结构上属于同一层级
- 上层往往被迫显式区分

组合模式解决的不是“如何访问文件”，而是：

如何统一表达结构层级。

3. 组合模式在文件系统中的核心矛盾

文件系统中存在一个天然矛盾：

结构上：

- 文件和目录都是“文件系统节点”

行为上：

- 文件不可包含子节点
- 目录可以包含子节点

客户端视角：

- 很多操作希望对“节点”统一处理

例如：

- 删除一个路径
- 打印一个路径树
- 查找某个文件
- 统计某个路径的大小

这些操作本质是：

对“节点”进行操作，而不是对“文件”或“目录”分别操作。

组合模式的价值就在这里：

让“节点”成为第一类抽象。

4. 抽象转化：从 API 到结构

FatFs 原生提供的是：

- 操作函数
- 路径字符串
- 类型标志

这是一种“过程式视角”。

组合模式引入的是：

结构视角。

我们不再说：

- “打开这个路径”

- “判断是不是目录”

而是说：

- “对这个节点执行操作”

这种转化是抽象层级的提升：

原始思维	模式思维
处理路径	处理节点
判断类型	类型内聚
分支控制	递归分发
API 调用	对象协作

5. 结构 (Structure)

文件系统天然符合组合模式结构：

```
Component (文件系统节点)
├─ Leaf (文件)
└─ Composite (目录)
```

这个结构的关键不是继承关系，而是：

统一抽象 + 递归组合。

目录可以包含：

- 文件 (Leaf)
- 目录 (Composite)

这使得结构无限递归。

6. 参与者 (Participants)

6.1 Component —— 文件系统节点

概念上代表：

一个可被访问的资源单元。

它定义统一操作：

- 查询信息
- 执行访问
- 可能的遍历能力

它不关心内部是否有子节点。

6.2 Leaf —— 文件

- 没有子节点
- 是递归终止点
- 承担具体操作

6.3 Composite —— 目录

- 持有子节点
- 执行操作时向下传播
- 是结构扩展的来源

6.4 Client —— 上层系统

Client 的理想状态是：

只知道“节点”，不知道“文件或目录”。

例如：

- CLI 的 `tree`
- 资源管理器
- 日志清理模块

7. 关键机制：递归一致性

组合模式真正的核心不是“统一接口”，而是：

递归一致性（Recursive Uniformity）。

含义是：

- 对组合对象的操作，会自动应用到其所有子对象。
- 操作逻辑在结构中自然传播。

在文件系统中：

- 删除目录 → 删除其所有子节点
- 打印目录 → 打印其所有子节点
- 统计目录大小 → 汇总其子节点大小

这种递归结构与组合模式完全一致。

8. 为什么文件系统是组合模式的经典案例？

因为它具备组合模式的三个必要条件：

① 部分-整体层次

目录包含文件和子目录。

② 统一抽象

文件和目录都是“文件系统节点”。

③ 递归传播操作

对目录的操作会向下传播。

这不是人为设计出来的结构，而是：

文件系统的自然结构。

9. 模式带来的架构意义（嵌入式视角）

在嵌入式系统中，引入组合模式后，文件系统从：

一组函数接口

变成：

一棵资源树。

这带来三个架构级收益：

① 系统结构清晰

资源管理从“分散调用”变为“结构化遍历”。

② 扩展性增强

可以自然加入：

- 多卷挂载
- 虚拟目录
- 只读资源节点
- 设备节点

而客户端不变。

③ 与其他系统统一

文件系统树可以与：

- 设备树
- 配置树
- 资源树

统一为同一种结构表达。

这在嵌入式架构设计中非常重要。

10. 本质总结

组合模式在 FatFs 场景中的本质不是：

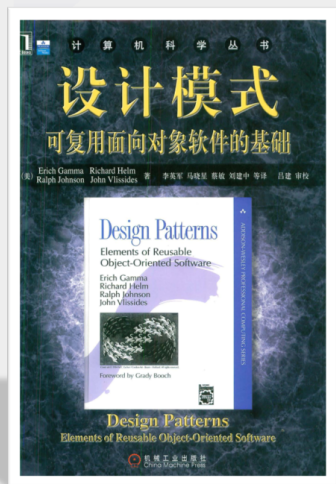
- 包装 API
- 写统一接口
- 减少 if 判断

而是：

把文件系统从“过程式操作集合”提升为“递归结构模型”。

它改变的不是代码形式，而是：

- 抽象层级
- 思维方式
- 系统结构表达



设计模式

面向嵌入式软件开发



针对C语言及嵌入式软件开发的
伴读教程，陪伴学习设计模式这一
进阶知识点。

Decorator（装饰模式）

1 Intent（意图）

动态地给对象附加额外职责。

Decorator 提供了一种 **比继承更灵活的功能扩展方式**。

通过在对象外部包裹新的对象，可以在不改变原对象接口的情况下扩展其行为。

在嵌入式系统中，可以理解为：

在核心模块外部逐层附加功能模块，使行为能够按需叠加，而无需修改原始模块。

2 Also Known As（别名）

Wrapper（包装器）

3 Motivation（动机）

在系统设计中，经常需要为某个模块增加附加功能，例如：

- 增加日志
- 增加校验
- 增加加密
- 增加统计
- 增加缓存

如果通过继承扩展功能，可能会产生大量组合类。例如：

- UARTWithCRC

- UARTWithLog
- UARTWithCRCAndLog
- UARTWithEncryptAndCRC

当功能组合增多时，类的数量会迅速膨胀。

Decorator 通过 **对象组合** 的方式解决这个问题。

每个增强功能独立实现，通过包装对象的方式逐层叠加。

例如：

```
Log
↓
Encrypt
↓
CRC
↓
UART
```

这样系统可以灵活组合不同功能，而无需创建大量派生类。

4 Applicability（适用性）

Decorator 适用于以下情况：

1 需要在运行时动态增加对象职责

功能可能按不同组合出现，而不希望在编译期固定。

2 希望避免大量子类

当功能组合较多时，继承会导致类型数量爆炸。

3 需要在不修改原对象的情况下扩展功能

Decorator 可以在保持接口不变的情况下增强行为。

4 多个增强功能可以独立实现

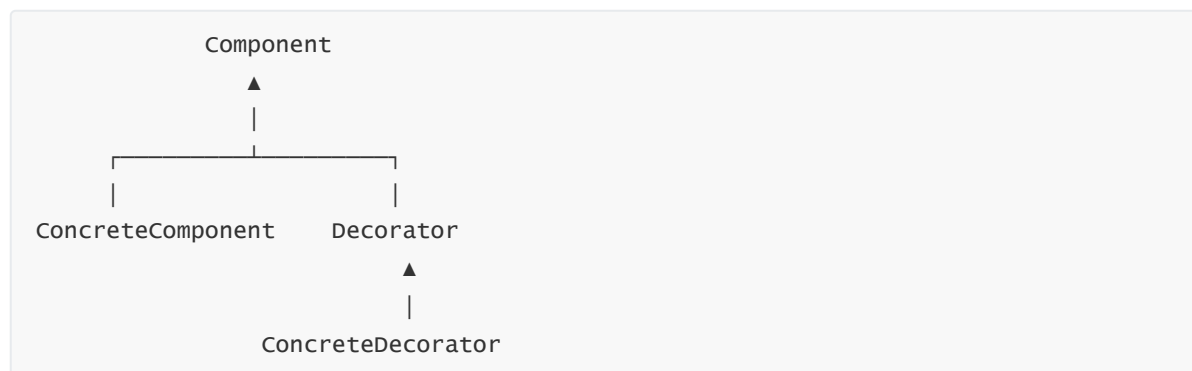
每个功能模块可以单独开发，并按需组合。

在嵌入式系统中，典型场景包括：

- 通信发送增强（CRC / 加密 / 日志）
 - 存储访问增强（缓存 / ECC / 加密）
 - 传感器数据处理（滤波 / 校准 / 平滑）
 - 日志输出增强（时间戳 / 颜色 / 异步缓冲）
-

5 Structure (结构)

Decorator 模式包含以下基本结构：



Decorator 与 Component 具有相同接口，同时内部持有一个 Component 对象。

因此系统可以形成 **递归包装结构**。

6 Participants (参与者)

Component

定义对象接口。

客户端通过该接口访问对象。

ConcreteComponent

实现基础功能的对象。

它是被装饰的核心对象。

Decorator

实现 Component 接口，并持有一个 Component 对象。

请求会被转发给该对象。

ConcreteDecorator

在转发请求时附加新的职责。

不同装饰器提供不同的增强行为。

7 Collaborations (协作)

客户端通过 Component 接口访问对象。

一个 ConcreteDecorator 接收请求后：

1. 执行自身增强逻辑
2. 将请求转发给内部的 Component

3. Component 继续处理请求

多个 Decorator 可以嵌套形成调用链。

例如：

```
Client
↓
Decorator A
↓
Decorator B
↓
ConcreteComponent
```

如果存在前后处理逻辑，则调用过程可能为：

```
A before
B before
Core operation
B after
A after
```

8 Consequences (效果)

Decorator 模式带来以下影响。

优点

1 比继承更灵活

功能可以通过组合动态叠加，而不需要创建新的子类。

2 支持功能组合

不同增强功能可以按需组合。

3 遵循开闭原则

系统可以在不修改原对象的情况下增加新功能。

4 职责划分清晰

每个 Decorator 只负责一个增强行为。

缺点

1 对象结构更复杂

系统可能形成较深的装饰链。

2 调试难度增加

行为分布在多个装饰层中。

3 行为顺序敏感

某些装饰器的执行顺序可能影响结果。

4 可能增加运行开销

在嵌入式系统中，多层调用可能增加函数调用成本。

9 Implementation（实现）

Decorator 的实现通常包含以下关键点：

1 统一接口

所有组件必须遵循同一接口。

2 装饰对象持有同接口对象

Decorator 内部必须保存一个 Component 引用。

3 请求转发

Decorator 在执行自身逻辑后，将请求转发给内部对象。

4 可递归组合

Decorator 本身也是 Component，因此可以继续被包装。

在嵌入式 C 中，这种结构通常通过：

- 接口结构体
- 函数指针表
- 对象组合

来实现。



设计模式

面向嵌入式软件开发



针对C语言及嵌入式软件开发的
伴读教程，陪伴学习设计模式这一
进阶知识点。

外观模式（Facade Pattern）——嵌入式讲义

1 意图（Intent）

为子系统的一组接口提供一个统一的高层接口，使子系统更易使用。

嵌入式理解

在嵌入式系统中：

- 子系统通常是：
 - 模块组合（Sensor / Display / Comm）
 - 系统服务（Power / RTOS / FileSystem）

👉 Facade 的作用：

把“复杂调用流程”收敛为“简单 API”

2 动机（Motivation）

✗ 问题场景（无 Facade）

例如温控系统：

```
temp = sensor->readValue();
display->showValue(temp);
actuator->setSpeed(speed);
comm->send(data);
```

问题：

- 上层必须知道所有模块
- 必须掌握调用顺序
- 耦合严重
- 难以维护

✓ 引入 Facade

```
Thermostat_RunCycle();
```

内部：

- 读取传感器
- 更新显示
- 控制风扇
- 发送数据

✓ 核心变化

维度	改变
接口数量	多 → 少
复杂度	分散 → 集中
调用者负担	高 → 低

3 适用性 (Applicability)

在嵌入式中，以下场景应使用 Facade：

✓ 1. 子系统复杂

例如：

- Power 管理（时钟 + 外设 + 中断）
- 网络协议栈（TCP/IP）

✓ 2. 调用顺序固定

例如：

初始化 → 采集 → 处理 → 输出

✓ 3. 希望解耦上层

- UI / 业务层 不关心底层细节
- 仅使用简单 API

✓ 4. 分层架构

```
Application
  ↓
Facade ← 推荐放置位置
  ↓
Driver / HAL / Middleware
```

4 结构 (Structure)

标准结构 (嵌入式映射)

```
Client
  ↓
Facade
  ↓
Subsystem A (Sensor)
Subsystem B (Display)
Subsystem C (Comm)
Subsystem D (Actuator)
```

嵌入式代码结构示例

```
// Facade
void Thermostat_RunCycle();

// 子系统
sensor_read();
display_show();
fan_set_speed();
comm_send();
```

5 参与者 (Participants)

1. Facade (外观)

- 提供统一接口
- 封装子系统调用
- 管理调用顺序

👉 示例:

```
void Power_sleep();  
void Device_Start();
```

2. Subsystem Classes (子系统)

- 实现具体功能
- 不知道 Facade 存在

👉 示例:

```
sensor.c  
display.c  
comm.c  
driver.c
```

3. Client (客户端)

- 只调用 Facade
- 不直接操作子系统

6 协作 (Collaborations)

调用流程

```
Client → Facade → Subsystems
```

嵌入式执行链

```
main()  
↓  
Thermostat_RunCycle()  
↓  
sensor_read()  
display_show()  
fan_control()  
comm_send()
```

关键点

- Facade 控制流程
- Client 不参与调度

7 结果 (Consequences)

✓ 优点

1. 降低耦合

- Client 不依赖子系统

2. 简化接口

```
Power_Sleep(); // vs 多个底层调用
```

3. 提高可维护性

- 修改子系统不影响上层

4. 明确分层

- 提供清晰架构边界

✗ 缺点

1. 可能成为“上帝类”

```
System_DoEverything();
```

2. 灵活性降低

- 高级用户可能需要绕过 Facade

8 实现 (Implementation)

✓ 1. 不强制结构形式

在嵌入式中可以是：

- 普通函数

- struct + 函数指针
- 模块接口

👉 关键是“接口层级”，不是语法

✓ 2. Facade 不应过重

建议：

- 按模块拆分 (Power / Device / Net)
- 避免单一巨大接口

✓ 3. 可结合其他模式

常见组合：

模式	作用
Abstract Factory	创建子系统
Singleton	管理全局资源
State	内部状态控制

✓ 4. 推荐结构（嵌入式）

```
Facade
  ↓
Service
  ↓
Driver / HAL
```

9 示例

示例1：Power 管理（经典）

```
void Power_Sleep();
void Power_Wakeup();
```

示例2：LCD 驱动（最常见）

```
LCD_Init();
LCD_DrawPixel();
LCD_Clear();
```

内部：

- SPI
- GPIO
- DMA

示例3：设备控制层

```
Device_Start();  
Device_Process();  
Device_Stop();
```

总结（嵌入式视角）

外观模式不是某种接口形式，而是一种系统边界设计方法。

✓ 核心本质

- 隐藏复杂子系统
- 提供统一入口
- 降低调用复杂度

✓ 在嵌入式中的典型落点

- Power 管理
- LCD / 外设驱动
- 文件系统 / 网络
- 设备控制层

✓ 一句话记忆

Facade = 让系统“更好用”



设计模式

面向嵌入式软件开发



针对C语言及嵌入式软件开发的
伴读教程，陪伴学习设计模式这一
进阶知识点。

一、Intent（意图）

在内存受限的系统中，**用共享数据的方式表示大量小对象**，避免重复存储。

二、Motivation（动机）

系统中存在大量“看起来是多个对象”，但实际上：

- 数据结构相同
- 大部分字段相同
- 只有少量差异

典型问题：

```
struct Device {
    uint8_t type;
    uint16_t p1;
    uint16_t p2;
};

struct Device dev[500];
```

如果：

- type 只有几种
- p1/p2 取值有限

那么：

👉 **绝大多数内存是在重复存同样的数据**

关键切入点：

一个“对象”的数据，并不一定都需要独占

三、Applicability（适用条件）

适用于以下场景：

1. 对象数量很大

- 通道
 - UI元素
 - 字符
 - 状态节点
-

2. 数据重复率高

- 配置离散
 - 参数组合有限
 - 模式数量远小于对象数量
-

3. 可以拆分状态

对象可以拆成：

- 共享部分（不随对象变化）
 - 变化部分（运行时确定）
-

4. 不依赖对象“身份”

系统只关心：

- 数据内容
- 行为结果

而不是“这是第几个对象”

四、Structure（结构）

系统由三部分组成：

1. 共享数据池

```
const struct Config configs[];
```

- 存放所有可复用的数据
 - 通常放在 Flash
-

2. 引用（索引）

```
uint8_t index;
```

- 每个对象不再存数据
 - 只存“指向哪一份共享数据”
-

3. 使用接口

```
cfg = &configs[index];
```

- 在使用时取出共享数据
 - 外部状态通过参数传入
-

整体结构：

```
对象 = index + 外部状态  
数据 = configs[]
```

五、Participants（参与角色）

1. 共享数据（Flyweight）

```
const struct Config {  
    uint8_t type;  
    uint16_t p1;  
    uint16_t p2;  
};
```

特点：

- 不可修改
 - 可被多个对象引用
-

2. 数据池（共享集合）

```
const struct Config configs[];
```

特点：

- 唯一存储位置
- 按索引访问

3. 使用者

```
cfg = &configs[idx];
```

负责：

- 提供索引
- 提供外部参数

六、Collaborations（协作关系）

运行流程：

1. 对象持有索引（而不是完整数据）
2. 使用时通过索引访问共享数据
3. 外部状态通过函数参数传入

示意：

```
index → configs[index] → 使用 + 外部参数
```

关键点：

共享数据不包含上下文信息，所有变化由调用方提供

七、Consequences（效果）

优点

1. 大幅减少内存占用

- 从 N 份数据 → 1 份 + N 个索引

2. 用 Flash 替代 RAM

- 共享数据可放只读区

3. 数据一致性高

- 所有对象引用同一份数据

代价

1. 外部状态必须显式管理

```
draw(x, y, index);
```

- 容易遗漏或传错

2. 访问间接化

```
cfg = &configs[idx];
```

- 可读性下降

3. 设计复杂度增加

- 需要拆分状态
- 需要设计共享粒度

八、Implementation（实现要点）

1. 优先使用 const 数据

```
const struct Config configs[];
```

2. 用索引代替指针

```
uint8_t idx;
```

- 更节省空间
- 更安全

3. 外部状态不入结构体

正确：

```
draw(cfg, x, y);
```

错误：

```
struct {  
    Config cfg;  
    int x, y;  
};
```

4. 控制共享粒度

- 太细：收益小
- 太粗：灵活性差

九、Example（嵌入式示例）

```
typedef struct {  
    uint8_t mode;  
    uint16_t period;  
    uint8_t duty;  
} LedConfig;  
  
const LedConfig configs[] = {  
    {0, 0, 0},  
    {1, 500, 50},  
    {2, 1000, 30}  
};  
  
uint8_t led_idx[100];
```

使用：

```
void led_run(int id) {  
    const LedConfig* cfg = &configs[led_idx[id]];  
    run_led(cfg->mode, cfg->period, cfg->duty);  
}
```

十、总结

享元模式的核心不在“对象”，而在：

把重复数据从对象中剥离出来，转为共享存储

在嵌入式系统中，它通常表现为：

👉 查表 + const 数据 + 索引访问 + 外部参数



设计模式

面向嵌入式软件开发



针对C语言及嵌入式软件开发的
伴读教程，陪伴学习设计模式这一
进阶知识点。

一、意图 (Intent)

在嵌入式系统中：

为硬件资源或关键对象提供一个受控访问入口。

核心：

- 所有访问必须经过代理层
- 代理层决定访问是否发生
- 调用方不关心底层细节

二、动机 (Motivation)

直接访问硬件的问题主要集中在三点：

1. 资源稀缺

- 总线共享 (I2C / SPI)
- Flash 擦写受限
- RAM 紧张

👉 需要统一控制访问

2. 状态复杂

- 未初始化 / 就绪 / 忙 / 错误

👉 状态不能分散在各处判断

3. 并发与成本

- 多任务竞争资源
- 外设访问有延迟 / 开销

👉 需要统一仲裁 + 控制访问频率

三、适用性 (Applicability)

适用于资源具备以下特征：

- 有状态 (init / busy / error)
- 有成本 (慢 / 限次数)
- 有竞争 (多任务访问)

常见需求：

- 状态检查
 - 延迟初始化
 - 缓存数据
 - 并发控制
-

四、结构 (Structure)

业务层 → 代理层（控制入口） → 驱动/硬件

结构要点

1. 接口稳定

业务层只面对统一接口

2. 状态集中

代理层持有：

- 状态 (init / busy / error)
- 缓存
- 同步控制

👉 状态不再分散

3. 资源归属明确

初始化、缓存、释放统一在代理层

五、参与者 (Participants)

1. 接口

定义可操作能力（函数接口）

2. 真实对象

- 直接访问硬件
 - 不负责状态控制和并发
-

3. 代理（核心）

负责：

- 状态管理
 - 访问控制
 - 缓存与同步
-

4. 业务层

只调用接口，不接触硬件

六、协作 (Collaborations)

基本流程：

1. 业务层调用接口
 2. 代理层判断状态（init / busy 等）
 3. 决定是否访问真实对象
 4. 返回结果（可能来自缓存）
-

👉 关键点：代理决定是否触发真实访问

七、效果 (Consequences)

✓ 优点

- 状态集中
 - 资源可控
 - 并发安全
 - 降低耦合
-

✗ 代价

- 增加一层调用
 - 占用额外内存 (状态 / 缓存)
 - 状态管理复杂度上升
-

八、实现要点 (Implementation)

1. 状态集中

- init_flag
 - busy_flag
 - error_flag
 - cache_valid
-

2. 内存控制

- 优先静态分配
 - 控制缓存大小
-

3. 访问策略

明确：

- 是否缓存
 - 是否阻塞
 - 是否允许并发
-

九、典型应用

- 传感器访问 (缓存数据)
- Flash / EEPROM (控制访问频率)
- 通信模块 (状态管理 + 节流)
- 多任务共享外设 (统一加锁)

总结

代理模式 = 给硬件访问加一个“统一控制入口”，把状态、资源、并发全部收口管理。



设计模式

面向嵌入式软件开发



针对C语言及嵌入式软件开发的
伴读教程，陪伴学习设计模式这一
进阶知识点。

责任链模式（Chain of Responsibility）

一、意图（Intent）

在嵌入式 C 里，责任链的核心不是“对象一个接一个”，而是：

把同一入口收到的请求交给一串处理节点依次试判和接管，应用层不直接决定该找哪个模块、哪个驱动、哪个协议分支。

它控制的是三件事：

- 谁有资格先看这个请求
- 谁有资格真正接管它
- 当当前节点不处理时，请求按什么顺序继续向后传递

所以从嵌入式角度看，责任链首先是在控制**请求分派权**，其次是在控制**状态判断顺序**。

二、动机（Motivation）

嵌入式系统里最常见的坏味道之一，就是把“谁来处理请求”直接写死在发送方里：

- `if/else` 决定交给哪个模块
- 按状态位判断哪个模块当前可用
- 按优先级手写多层回退逻辑
- 在多个文件里散落“如果 A 不行就试 B”

这种写法在 PC 软件里已经不好维护，在嵌入式里问题会更明显：

1. 分派规则和业务动作混在一起

应用层本来只该表达“我要处理这个事件”，结果却顺手承担了：

- 状态裁决
- 模块选择
- 降级路径
- 默认兜底

这样一来，应用层会知道太多驱动层和中间层细节。

2. 可裁剪性差

很多嵌入式项目需要按芯片型号、资源等级、SKU 做裁剪。

如果分派顺序写死在业务层，那么删掉一个模块、替换一个协议处理器、调整优先级，往往都要改上层代码。

3. 时序与资源判断容易散开

真实系统里，一个请求能否处理，通常不只取决于“功能上支持不支持”，还取决于：

- 模块是否初始化完成
- 当前是否 busy
- 总线是否占用
- 电源域是否已打开
- 当前上下文是不是 ISR

这些判断若散落在不同调用点，系统会越来越难排查。

4. 默认行为很难统一

很多开源项目和厂家 SDK 的默认做法，都是先提供**统一入口**，再让若干候选处理节点按顺序试判，而不是让发送方直接绑定某个具体实现。

责任链就是把这部分“谁接管请求”的职责，从发送方挪到链本身。

三、适用性（Applicability）

当系统满足下面这些条件时，责任链就很合适：

- **有统一入口**：请求都从 UART、USB、事件中心、协议层入口进入
- **有多级判断**：是否支持、是否就绪、是否允许、是否降级都要判断
- **有顺序要求**：必须先试主处理器，再试备用处理器，再试兜底处理器
- **需要可扩展**：后续可能新增一个处理节点，但不想改发送方
- **需要可裁剪**：不同产品线启用不同节点组合

嵌入式里常见的判断语句不是“这个模式像不像”，而是：

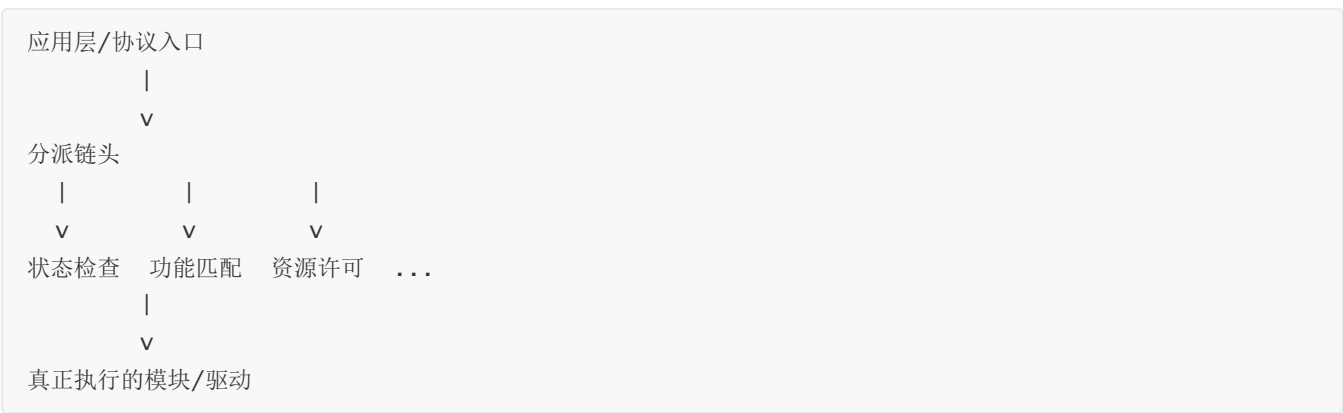
请求有多个候选接收者，且接收权要按顺序在运行时决定。

四、结构（Structure）

从嵌入式 C 的视角，可把它看成下面这条路径：



若把工程层次写清楚，通常会更像这样：



每个节点只做两件事：

1. 判断自己现在是否应该接管
2. 若不接管，就把请求交给下一个节点

在 C 里，链不一定非要真用链表。

常见实现有三类：

- 结构体 `next` 指针串链
- 静态处理表按顺序遍历
- 编译期注册段 + 运行期顺序遍历

五、参与者（Participants）

1. Handler（处理节点接口）

它定义统一的处理入口。

在嵌入式 C 里，通常体现为：

- 统一函数原型
- 统一节点结构体

- 统一返回值约定

例如概念上会有这样的语义：

- `HANDLED`：我已经接管
- `PASS`：我不处理，继续传递
- `ERROR`：发现错误，直接终止

这里最重要的不是“接口长什么样”，而是：

链上的所有节点都必须用同一种处理语义说话。

2. ConcreteHandler（具体处理节点）

具体节点负责自己的那一段判断逻辑，例如：

- 只处理某类命令字
- 只处理某类设备地址
- 只在模块 ready 时接管
- 只在资源未占用时接管

它不应该知道全局所有规则，只需要知道：

- 我的受理条件
- 我的执行动作
- 不处理时把请求交给谁

3. Request / Context（请求与环境）

嵌入式里很多链式处理都离不开上下文，因此通常会有：

- 请求本体
- 当前解析位置
- 调用上下文（任务态 / ISR）
- 权限或来源信息
- 临时状态标志

责任链是否稳定，往往取决于这份上下文是否设计清楚。

4. Client（请求发起方）

发起方只负责把请求送进链头。

它最好只知道“统一入口”，而不知道后面有几个节点、谁先谁后、谁会最终处理。

六、协作（Collaborations）

责任链在嵌入式系统中的执行路径一般是：

1. 应用层、协议层或中断后半部生成一个请求

2. 请求被送入链头
3. 当前节点检查自己是否满足接管条件
4. 若满足，则在当前节点完成动作并结束
5. 若不满足，则返回“继续传递”
6. 后继节点重复同样流程
7. 最终由某个节点处理，或由兜底节点给出默认结果

这里真正关键的是：

发送者只负责“提交请求”，接管决策沿链传递。

这比在业务层写一堆“先试 A，再试 B，再试 C”更适合长期维护。

七、效果（Consequences）

收益

1. 发送方与具体处理模块解耦

应用层不再知道到底是哪个驱动、哪个协议子模块、哪个回退分支最终接管。

2. 处理顺序可集中管理

优先级、降级顺序、兜底行为都收敛到链中，而不是散落在业务代码里。

3. 更适合裁剪和扩展

新增一个节点、删掉一个节点、交换两个节点顺序，通常不需要改请求发送方。

4. 更符合嵌入式的资源准入逻辑

很多处理并不是“功能上能不能做”，而是“当前状态下允不允许做”。

责任链很适合承载这种逐级准入判断。

代价

1. 跟踪路径会变长

链长了以后，要查“为什么最后是这个节点接管”会变复杂。

2. 返回语义必须统一

一旦不同节点对 `PASS` / `HANDLED` / `ERROR` 的理解不一致，链就会非常难调。

3. 可能引入额外遍历开销

在极端高频路径上，链太长会带来可见的时间成本。

八、实现要点 (Implementation)

1. 优先静态组织，少做动态拼链

大多数 MCU 项目更适合：

- 静态数组
- 编译期注册表
- 固定顺序链

这样更容易控制 RAM、启动顺序和可预测性。

2. 明确返回值协议

务必在接口层规定清楚：

- 什么叫“我已处理”
- 什么叫“继续传递”
- 什么叫“直接失败”
- 出错后是否允许后继继续尝试

3. 不要让节点偷改全局状态

链式处理最怕某个节点在返回 `PASS` 前已经改了一半系统状态，后续节点又继续处理，最后留下脏状态。

4. 注意 ISR 与任务上下文边界

若请求从中断进入链，链上节点最好只做轻量判定，把真正重动作延后到任务上下文。

5. 兜底节点要明确

链尾到底是：

- 返回未支持
- 返回参数错误
- 返回默认处理
- 打日志并丢弃

一定要在设计阶段讲清楚。

九、典型应用

下面这些都不是为了讲模式硬凑出来的例子，而是很多开源项目和厂家默认就采用的组织方式：

1. USB 设备栈的控制请求分派

很多开源 USB Device 栈和厂家 USB 中间件，都会把 EP0 的 Setup 请求先送入统一入口，再依次判断：

- 标准请求
- 类请求
- 厂商自定义请求
- 用户兜底处理

哪个节点先匹配成功，哪个节点就接管请求。
这本质上就是责任链。

2. 内核/底层的通知链机制

像 Linux kernel 的 notifier 机制，本质上也是事件进入统一入口后，沿已注册处理者顺序通知，某些场景下还能中止后续传播。

3. GUI / 输入事件冒泡

很多嵌入式图形框架和厂家 HMI 方案里，一个输入事件会先交给当前控件，再按父层级向上冒泡，直到某一级消费掉事件。

这也是责任链非常典型的形态。

4. 资源准入检查链

很多产品里，一次“执行命令”并不会立刻碰硬件，而是先经过：

- 参数合法性检查
- 当前模式检查
- 电源域检查
- 外设 busy 检查
- 最终驱动执行

这类多级准入非常适合责任链。

总结

责任链在嵌入式 C 里的本质，就是把“谁来接管请求”的决定权从发送方拿出来，交给一条按顺序试判的处理链。



设计模式

面向嵌入式软件开发



针对C语言及嵌入式软件开发的
伴读教程，陪伴学习设计模式这一
进阶知识点。

命令模式（Command）

一、意图（Intent）

在嵌入式 C 里，命令模式的核心不是“面向对象封装动作”，而是：

把一次操作封成一个可传递、可排队、可延后执行的命令单元，让触发者只负责提交，真正执行者在合适的上下文里落地。

它控制的是：

- 请求以什么数据结构被表示
- 请求由谁触发、由谁真正执行
- 请求在什么时候、在哪个上下文里执行

所以命令模式首先是在控制**执行时机**，其次是在控制**调用者与执行者之间的耦合**。

二、本质 / 动机（Motivation）

嵌入式项目里经常遇到这种场景：

- ISR 想触发一个复杂动作
- 应用层想请求一次外设操作，但不能立刻做
- 多个任务都想访问同一资源，需要串行化
- 某个动作要排队、记录、重试或延迟执行

如果直接写成“谁触发谁就直接调函数”，问题会马上出现。

1. 触发上下文和执行上下文不一致

例如：

- 按键中断只适合上报事件，不适合直接刷屏或写 Flash
- 协议解析线程可以接受命令，但实际硬件访问必须在专门任务里串行执行

这时“谁提出动作”和“谁执行动作”本来就不该是同一个角色。

2. 共享资源需要统一串行化

SPI、I2C、Flash、显示控制器、无线模组这些资源，往往都不适合被多个调用点直接并发操作。

若所有地方都直接调驱动，资源竞争会非常难收敛。

3. 请求需要排队或记录

真实产品里，经常需要：

- 稍后执行
- 放入队列
- 失败后重试
- 记录历史
- 统一做超时和统计

直接函数调用并不擅长表达这些要求。

4. 很多开源 RTOS 和厂家 SDK 默认就这么做

它们并不是让业务层随时随地直接碰底层资源，而是把“要做什么”先变成一个命令或工作项，再交给：

- 服务任务
- 工作队列
- 事件循环
- 后台线程

统一执行。

这正是命令模式在嵌入式里的真实价值。

三、适用性（Applicability）

命令模式特别适合这些条件：

- **需要跨上下文执行**：ISR 触发、线程执行
- **需要延迟执行**：现在提请求，稍后再做
- **需要串行访问共享资源**：一个后台实体统一执行
- **需要排队**：请求先缓存，再按顺序处理

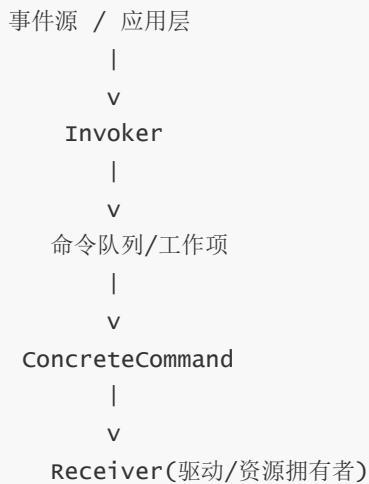
- **需要记录/重放**：测试、日志、恢复、调试
- **需要把触发者与具体实现解耦**：调用方只知道“提交命令”

一句话判断：

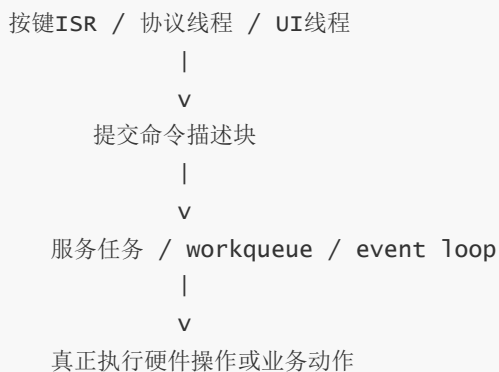
如果系统关心的不只是“做什么”，还关心“何时做、在哪做、按什么顺序做”，命令模式通常就值得考虑。

四、结构（Structure）

从嵌入式 C 的视角，可以把它看成：



更贴近工程实现时，常见关系是：



在这里：

- 提交方不直接碰资源
 - 执行方统一在受控上下文运行
 - 命令本身只是“这次要做什么”的数据载体
-

五、参与者（Participants）

1. Command（命令接口或命令描述）

在嵌入式 C 中，它往往不是一个“类”，而是一种统一的数据表达。

通常会包含：

- 命令 ID
- 参数区
- 目标对象或句柄
- 可选回调
- 状态位
- 超时/时间戳

也就是说，命令首先要把“这次动作”描述清楚。

2. ConcreteCommand（具体命令）

具体命令表示一类明确动作，例如：

- 启动一次采样
- 停止某路 PWM
- 提交一次 Flash 擦写
- 切换某个工作模式

它不一定非要做成很多结构体类型。

在 C 里，更常见的是：

- 一个统一命令结构体 + `cmd_id`
- 配合参数联合体或字段集

3. Receiver（真正执行者）

Receiver 才是真正拥有资源、真正落地动作的地方。

例如：

- Flash 服务任务
- 显示刷新线程
- 总线仲裁层
- 某个具体驱动模块

命令不负责实现细节，它只是把请求组织好并交出去。

4. Invoker（触发器 / 提交器）

Invoker 的职责是：

- 接收外部动作请求
- 保存或转发命令
- 在合适时机触发执行

它不需要知道底层实现细节。

它只需要知道“把什么命令交出去”。

5. Client（发起方）

Client 决定：

- 创建什么命令
- 填什么参数
- 交给哪个 Invoker

在嵌入式里，Client 很可能是：

- 一个 ISR 后半部
- 一个应用任务
- 一个 shell 命令处理函数
- 一个状态机节点

六、协作（Collaborations）

命令模式在嵌入式里的常见执行路径是：

1. 某个上下文产生动作请求
2. Client 组织命令描述块
3. 命令交给 Invoker
4. Invoker 将命令排队、缓存或立即转发
5. Receiver 在自己的上下文中取到命令
6. Receiver 真正执行硬件动作或业务动作
7. 执行完成后可选择上报结果、记录日志或触发下一条命令

这条路径表达的是：

请求的提出与请求的落地被拆开了。

这在嵌入式里非常重要，因为很多系统问题并不是“功能错了”，而是“上下文错了、时机错了、资源归属错了”。

七、效果（Consequences）

收益

1. 调用方与执行方解耦

业务层不需要知道：

- 真正谁执行
- 在哪个线程执行
- 资源什么时候可用
- 执行过程中怎么重试或排队

2. 执行时机可控

命令可以：

- 立刻执行
- 进入队列
- 延时执行
- 由后台线程统一执行

这对 ISR/线程边界、共享资源串行化都很有价值。

3. 天然适合日志、排队和重放

只要命令被表示成数据，就容易：

- 记录
- 转储
- 回放
- 统计
- 测试

4. 更容易做故障隔离

若某类资源必须由专门任务统一访问，那么把所有操作都表示成命令，会比四处直接调驱动安全得多。

代价

1. 引入额外数据结构和调度层

命令池、队列、状态管理都会带来额外 RAM 和复杂度。

2. 简单场景下会显得偏重

如果只是一个局部直接函数调用，没有跨上下文、没有排队、没有资源争用，那么命令模式可能不划算。

3. 需要明确命令所有权

命令由谁申请、谁填写、谁释放、失败后谁回收，这在 C 里必须讲清楚。

八、实现要点 (Implementation)

1. 优先设计“命令数据模型”

在嵌入式里，命令模式成不成立，关键常常不在接口，而在命令结构体是否清楚：

- 命令 ID
- 参数
- 返回路径
- 生命周期
- 所属资源

2. 优先静态分配

命令池、队列、工作项结构更适合静态或预分配方式，避免运行时碎片和不可控失败。

3. 把资源访问权限集中到 Receiver

不要让 Client 一边发命令，一边又偷偷直接碰底层资源。

否则命令模式只剩表面形式，资源仍是失控的。

4. 明确失败语义

队列满、资源忙、执行失败、超时、重复命令等情况都要有统一返回语义。

5. 注意命令上下文切换

如果命令从 ISR 投递到线程执行，接口必须保证：

- ISR 路径无阻塞
- 参数在切换后仍然有效
- 不把临时栈数据错误地交给后台线程

九、典型应用

下面这些都不是课堂虚构例子，而是大量开源系统和厂家 SDK 默认就采用的思路：

1. FreeRTOS 软件定时器服务任务

FreeRTOS 的软件定时器并不是谁启动谁就自己执行回调，而是通过定时器服务任务统一管理，回调也在这个受控上下文里执行。

这非常符合命令模式的思想：

- 外部发起“启动/停止/重载”请求

- 后台统一接收并执行

2. Zephyr 的 workqueue / k_work

很多 Zephyr 场景里，ISR 或普通线程并不直接做重活，而是提交 `work item`，由系统工作队列稍后执行。这就是非常典型的“命令提交”和“后台执行”分离。

3. Linux kernel workqueue / deferred work

Linux 内核大量使用 deferred work，让触发者先提交工作，再由合适上下文执行，避免在不合适的上下文里直接做重操作。

4. 厂家 SDK 里的总线服务任务

很多 MCU 厂家示例工程、模组 SDK、显示/存储中间件里，都会把 SPI、I2C、Flash 这类共享资源的访问变成队列请求，由统一线程串行处理。

这也是命令模式在产品代码里的高频落点。

总结

命令模式在嵌入式 C 里的本质，不是“把函数包一下”，而是把一次动作变成一个可排队、可延后、可跨上下文执行的命令单元。



设计模式

面向嵌入式软件开发



针对C语言及嵌入式软件开发的
伴读教程，陪伴学习设计模式这一
进阶知识点。

解释器模式（Interpreter）

一、意图（Intent）

在嵌入式 C 里，解释器模式要解决的不是“收到一个命令就分发出去”，而是：

当输入本身已经是一门小语言时，把它按约定语法拆开、理解，并得到可执行含义。

它控制的是：

- 输入到底由哪些语法单元组成
- 各个单元怎样组合成完整语义
- 解释时依赖哪些上下文状态

所以解释器模式真正面对的是**语法**，而不是单纯的“命令号分派”。

二、本质 / 动机（Motivation）

很多嵌入式系统最后都会长出一门“小语言”，只是规模大小不同。

典型来源有：

- 串口 shell
- AT 命令
- 调试命令行
- 启动脚本
- 测试脚本

- 规则配置
- 简化表达式

如果把这些输入全部手写成一堆 `strcmp()`、`if/else`、位置偏移判断，短期能跑，长期通常会出问题。

1. 语法规则会散掉

最开始也许只有几条命令，后面一旦出现：

- 子命令
- 可选参数
- 变量
- 条件语句
- 多条命令串接

零散的字符串判断会迅速失控。

2. 输入结构与执行逻辑混在一起

系统既要负责判断“这串字符是什么意思”，又要负责“实际执行什么动作”。这样一来，解析器、状态机、执行器会互相缠在一起。

3. 扩展成本越来越高

每新增一个新语法，都要改大量已有判断。

很容易出现：

- 旧命令被新规则误伤
- 某些路径漏了参数检查
- 错误提示不一致

4. 很多开源项目和厂家工具链默认就把它当“小语言”处理

无论是 bootloader 命令行、RTOS shell，还是模组 AT 命令，真正稳定的实现通常都不是“直接比较整行字符串”，而是先做：

- 词法拆分
- 参数解释
- 子命令树匹配
- 语义执行

这就是解释器模式在嵌入式中的典型落点。

三、适用性（Applicability）

解释器模式适合这些条件：

- **输入有语法**：不是一个命令码，而是一段有规则的文本或 token
- **规则相对稳定**：语法结构不会每天大改

- **要反复解释**：系统长期处理同一套语法
- **命令族逐渐增多**：存在主命令、子命令、参数组合
- **需要更好的可维护性**：希望“语法”和“执行”分层

一句话判断：

如果输入已经像一门小语言，而不是一个简单枚举值，就应该用解释器的视角去设计。

四、结构（Structure）

在嵌入式 C 场景里，可以把解释过程看成：

```
输入流
  |
  v
词法拆分 / 参数切分
  |
  v
语法节点匹配
  |
  v
语义解释
  |
  v
执行动作
```

若把角色写完整，常见关系是：

```
Input -> Token/Command Tree -> Expression -> Context -> Executor
```

这里要注意：

- 解释器并不一定非要构建复杂 AST
- 很多 MCU 项目更常见的是“命令树 + 参数解释 + 上下文执行”
- 只要输入被按一套语法理解，这仍然是解释器模式的范畴

五、参与者（Participants）

1. AbstractExpression（统一解释接口）

它定义“这个语法单元怎样解释”。

在 C 里，这通常表现为：

- 一组统一解释函数
- 命令节点回调接口
- 子命令解析器接口

重点不在面向对象外形，而在：

不同语法单元都按统一协议参与解释。

2. TerminalExpression（终结符）

它表示最小语法单位，例如：

- 一个关键字
- 一个数字
- 一个变量名
- 一个固定选项

它负责把最小单元解释成可以使用的值。

3. NonterminalExpression（非终结符）

它表示由多个语法单元组合成的结构，例如：

- 一条完整命令
- 一个带参数的子命令
- 一段条件表达式
- 一条脚本语句

它负责组合子规则，得到更完整的语义。

4. Context（上下文）

在嵌入式系统里，这个角色通常非常重要。

它往往包含：

- 当前输入位置
- 参数表
- 环境变量
- 当前设备状态
- 查找表
- 命令执行环境

解释器不是只靠字符串本身就能工作，往往还要依赖这些上下文。

5. Client（发起解释者）

Client 负责把输入交给解释系统，并在解释完成后触发对应动作。

它通常是：

- shell 主循环
 - CLI 收发线程
 - bootloader 命令入口
 - AT 命令入口
-

六、协作（Collaborations）

典型执行路径如下：

1. 系统收到一段输入
2. 先按规则切分出 token 或参数
3. 再根据语法结构找到对应节点或表达式
4. 每个节点在上下文中解释自己的含义
5. 多个子单元组合后得到完整语义
6. 最终交给执行层完成动作

这里最重要的点是：

系统不是“看见一串字符就直接调函数”，而是先按语法理解，再决定动作。

这就是解释器模式和普通命令分派的本质区别。

七、效果（Consequences）

收益

1. 语法结构更清楚

命令字、参数、子命令、变量、条件这些角色可以分层表达，而不是全堆在一堆字符串比较里。

2. 更适合扩展小语言

随着命令族变多、参数组合变复杂，解释器模式通常比手写分支更稳。

3. 语法与执行可以分层

“输入怎样理解”和“理解后做什么”不必完全混在一起。

4. 错误处理更一致

非法命令、参数缺失、语法错误、上下文不满足等问题，更容易统一处理。

代价

1. 文法一大就会变复杂

如果语法已经接近完整脚本语言，解释器层会迅速膨胀。

2. 增加了解析和上下文管理成本

在资源很紧的小 MCU 上，要控制实现规模。

3. 不适合特别简单的命令码场景

如果输入只是几个固定枚举值，没有语法结构，那么没必要把它做成解释器。

八、实现要点 (Implementation)

1. 先区分“命令分派”与“语法解释”

如果只是根据一个 ID 或一个函数表查找处理器，那更像普通分派；
如果要理解输入结构，那才是解释器模式的真正应用点。

2. 优先做清晰的 token/参数层

不要一上来就在业务代码里到处直接切字符串。
先把词法切分、参数提取、合法性检查层整理好，会让后续扩展轻松很多。

3. 上下文要可控

解释器若要访问设备状态、变量区、权限状态，必须明确这些状态由谁持有、何时有效。

4. 错误路径要单独设计

嵌入式命令系统里，错误信息本身就是调试能力的一部分。
语法错、参数错、状态错，最好能区分开。

5. 控制代码规模

MCU 项目要避免为了“讲模式完整”而引入过重的抽象。
很多时候：

- 命令树
- token 切分
- 参数解释
- 上下文执行

已经足够形成一套轻量解释器。

九、典型应用

下面这些都属于大量开源项目和厂家平台里的默认做法：

1. Zephyr Shell

Zephyr shell 默认就是命令树组织方式：根命令、静态子命令、动态子命令，再配合参数解析与执行。
它不是简单的字符串分发，而是一门轻量命令语言的解释过程。

2. ESP-IDF Console

ESP-IDF 的 console 组件提供命令注册、参数切分、参数解析和 REPL 组织能力。这本质上就是在解释一套交互式命令语言。

3. U-Boot 命令行

U-Boot 的命令行支持命令解析、环境变量、分号串接，甚至可以使用更强的 hush shell。这说明 bootloader 里经常不是单条命令分发，而是解释一门小型脚本语言。

4. 模组和设备的 AT 命令解析器

大量无线模组、通信模组和厂家调试口默认都使用 AT 风格语法：

- 命令字
- 参数
- 查询/设置/执行三种语义

这正是解释器模式最常见的工业落点之一。

总结

解释器模式在嵌入式 C 里的本质，就是把输入当成一门小语言来理解，而不是把它当成一个简单命令号来分派。



设计模式

面向嵌入式软件开发



针对C语言及嵌入式软件开发的
伴读教程，陪伴学习设计模式这一
进阶知识点。

迭代器模式（Iterator）

一、意图（Intent）

在嵌入式 C 里，迭代器模式的核心不是“做一个对象负责遍历”，而是：

在不暴露容器内部组织方式的前提下，按统一方式依次访问注册项、链表节点、设备表、观察者表或链接段条目。

它控制的是：

- 外部怎样顺序访问元素
- 容器内部怎样存放元素
- 遍历过程中允许哪些操作

所以迭代器模式解决的，是**访问方式统一与存储细节隐藏**的问题。

二、本质 / 动机（Motivation）

在嵌入式工程里，数据集合的内部表示经常不是普通数组，常见形态包括：

- 单链表 / 双链表
- 环形缓冲区
- 静态注册表
- linker section
- 设备实例表

如果客户端代码直接依赖这些内部结构，就会出现几个问题。

1. 遍历代码和存储实现绑死

一旦从数组改成链表、从链表改成链接段表，所有调用方都得跟着改。

2. 容器内部细节泄漏

客户端本来只想“挨个访问元素”，却被迫知道：

- 节点字段叫什么
- 头指针在哪里
- 结束条件怎么判断
- 删除时该怎么安全前进

这会导致上层代码越来越依赖底层布局。

3. 多种遍历需求难以统一

同一批元素往往会有不同访问策略：

- 正向遍历
- 逆向遍历
- 过滤遍历
- 安全删除遍历
- 只读遍历

如果把这些逻辑都塞到容器里，容器会越来越臃肿。

4. 开源内核、RTOS、厂家 SDK 大量默认采用统一遍历接口

像 Linux kernel 的 list 宏、Zephyr 的链表 API、观察者遍历接口、设备注册表遍历，本质上都在做同一件事：

让使用者按统一规则访问元素，而不必直接碰容器内部布局。

这就是迭代器模式在 C 工程里的真实形态。

三、适用性（Applicability）

迭代器模式适合这些条件：

- **容器内部实现可能变化**：数组、链表、注册段都可能替换
- **外部只想访问，不想依赖布局**
- **存在多种遍历策略**：正向、逆向、安全删除、过滤
- **希望统一使用方式**：不同集合用类似方式访问
- **希望降低上层对底层数据结构的耦合**

一句话判断：

如果系统的重点是“怎样稳定访问元素”，而不是“元素底层怎么存”，就该用迭代器思路。

四、结构（Structure）

在嵌入式 C 里，迭代器不一定长成 GoF 图里的对象层次。
更常见的是下面三种形态：

1. 游标结构体 + next/current 接口
2. foreach 宏 + 容器约定
3. 回调式遍历接口

把它抽象成关系，可以写成：

```
Aggregate(集合)
  |
  v
Iterator(游标/遍历规则)
  |
  v
Client 逐项访问元素
```

若把工程层级说清楚，更像是：

```
设备表 / 链表 / 注册段
  |
  v
统一遍历协议
  |
  v
业务层按顺序读元素
```

关键不是外形，而是职责拆分：

- 容器负责保存元素
- 迭代器负责定义访问顺序
- 客户端负责使用访问结果

五、参与者（Participants）

1. Iterator（迭代器 / 游标规则）

在嵌入式 C 里，Iterator 常体现为：

- 当前游标
- 获取当前元素的方法
- 进入下一个元素的方法
- 判断结束的方法

它定义了“怎么走”。

2. ConcreteIterator（具体迭代器）

它知道某个具体集合怎样前进，例如：

- 数组按索引前进
- 链表按 `next` 前进
- 双链表支持反向前进
- 链接段按地址范围前进

3. Aggregate（聚合 / 容器）

容器负责存数据。

在 C 工程里，它可能是：

- 一个链表头
- 一个设备表
- 一个注册段区间
- 一个 observer 集合

4. Client（客户端）

Client 不应该直接依赖容器内部表示。

它最好只知道：

- 如何开始遍历
- 如何拿当前项
- 如何前进
- 何时结束

六、协作（Collaborations）

迭代器模式的典型协作流程是：

1. Client 向集合请求一个遍历入口，或拿到约定好的遍历宏/接口
2. Iterator 定位到第一个有效元素
3. Client 读取当前元素
4. Iterator 前进到下一个元素
5. 重复以上过程直到结束

这里最重要的是：

客户端在“顺序访问元素”，但不需要知道容器内部长什么样。

这让底层从数组改为链表、从普通表改为注册段时，上层代码仍可以保持相对稳定。

七、效果（Consequences）

收益

1. 容器内部结构对外隐藏

客户端不用依赖底层节点布局、头尾指针和边界规则。

2. 遍历逻辑可以独立演化

不同遍历策略可以在不改容器的前提下扩展。

3. 客户端代码更统一

无论底层是链表、数组还是注册段，外部都能用相近方式访问。

4. 便于维护大型注册系统

设备表、命令表、观察者表、驱动表这些场景非常受益。

代价

1. 简单场景下可能显得多一层

如果只是遍历一个很小的局部数组，专门抽一层迭代协议可能过重。

2. 安全语义要单独约定

遍历时是否允许删除当前节点、是否允许并发修改、是否需要加锁，这些都要明确。

3. 某些复杂结构的遍历并不轻松

树、图、带回溯规则的集合，具体迭代器仍然可能很复杂。

八、实现要点（Implementation）

1. 在 C 里不要执着于“类式实现”

很多成熟项目直接使用：

- 宏
- 游标结构
- begin/end 区间
- 回调遍历

只要职责成立，它就是迭代器思路。

2. 先定义遍历语义，再定容器结构

要先说清：

- 起点是什么
- 终点是什么
- 前进规则是什么
- 是否允许遍历中删除
- 是否需要锁保护

3. 明确元素所有权

迭代器只是访问，不等于拥有元素。

遍历时拿到的是：

- 只读指针
- 可写指针
- 临时快照

要事先讲清楚。

4. 注意并发和 ISR 场景

如果集合可能被别的线程、中断或回调同时修改，必须明确安全策略。

5. 对静态注册表尤其有价值

嵌入式系统大量使用：

- 设备实例表
- 命令注册表
- 初始化表
- 观察者集合

这类场景特别适合统一迭代协议。

九、典型应用

下面这些都是很多开源项目和厂家默认组织方式中的高频例子：

1. Linux kernel 链表遍历宏

Linux 内核的 `list_for_each_entry()`、`list_for_each_entry_safe()` 等，本质上就是为 intrusive list 提供统一访问方式。

调用者顺序访问元素，但不直接展开底层链表细节。

2. Zephyr 的 `sys_slist` / `sys_dlist`

Zephyr 提供统一的链表 API，让模块在不暴露内部存储细节的前提下组织和遍历节点。

3. Zephyr zbus 的 `channel` / `observer` 遍历

zbus 既支持通道定义，也提供遍历 `channel`、`observer` 的相关接口。

这说明在一个消息总线系统里，“如何统一访问注册对象”本身就是重要设计点。

4. 设备模型和静态注册表遍历

很多 RTOS、bootloader、厂家 SDK 会把驱动、命令、初始化项放进静态注册表或链接段，再用统一方式遍历。

这就是迭代器模式在 C 工程里的另一种常见形态。

总结

迭代器模式在嵌入式 C 里的本质，就是把“遍历”从具体存储布局里分离出来，让客户端稳定地访问元素，而不用绑定底层数据结构。



设计模式

面向嵌入式软件开发



针对C语言及嵌入式软件开发的
伴读教程，陪伴学习设计模式这一
进阶知识点。

中介者模式（Mediator）

一、意图（Intent）

在嵌入式 C 里，中介者模式的核心不是“多一个中心对象”，而是：

把多个模块之间本来杂乱的直接调用，收拢到一个统一协调中心，让各模块只向中心汇报状态、提交消息或请求协作，而不直接彼此耦合。

它控制的是：

- 模块之间的信息交换路径
- 协作规则由谁集中决定
- 哪些动作需要被串行化、转发或延后

所以中介者模式本质上是在控制**模块协作关系**。

二、本质 / 动机（Motivation）

嵌入式系统一开始往往只有几个模块：

- 传感器
- 通信
- UI
- 电源管理
- 存储

当模块变多后，如果它们彼此直接调用，很快就会形成一张网：

- 传感器直接通知 UI
- UI 直接控制蜂鸣器
- 通信模块直接改状态机
- 电源管理直接回调多个业务模块

短期看似方便，长期却很危险。

1. 依赖关系迅速膨胀

当一个模块知道很多别的模块时：

- 初始化顺序难安排
- 替换模块很麻烦
- 单元测试很难做
- 复用困难

2. 协作逻辑分散

真正复杂的往往不是“某个模块自己做什么”，而是“它和别人怎么配合”。
若协作逻辑分散在各个模块内部，系统会越来越不可读。

3. 上下文和时序越来越难控

直接互调时，经常会出现：

- 在 ISR 里直接碰 UI 或通信栈
- 低优先级模块直接触发高代价动作
- 多个模块抢着改同一份系统状态

4. 很多开源 RTOS 和厂家 SDK 默认采用事件循环 / 软件总线 / 中央协调器

也就是说，它们并不鼓励模块之间互相强绑定，而是让模块：

- 发布事件
- 注册处理函数
- 通过中心总线传递消息
- 由统一协调层完成联动

这正是中介者模式在嵌入式中的自然落点。

三、适用性（Applicability）

中介者模式适合这些条件：

- **对象或模块很多**：存在多对多交互趋势
- **交互规则比模块本身更复杂**

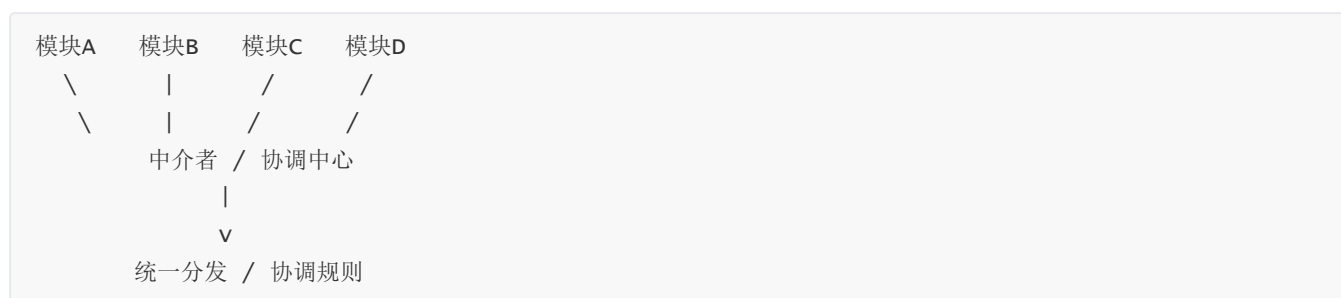
- **希望减少直接引用**：模块最好彼此不知道实现细节
- **需要统一调度上下文**：事件要在合适线程/循环中执行
- **协作规则经常变化**：流程会改，但基础模块相对稳定

一句话判断：

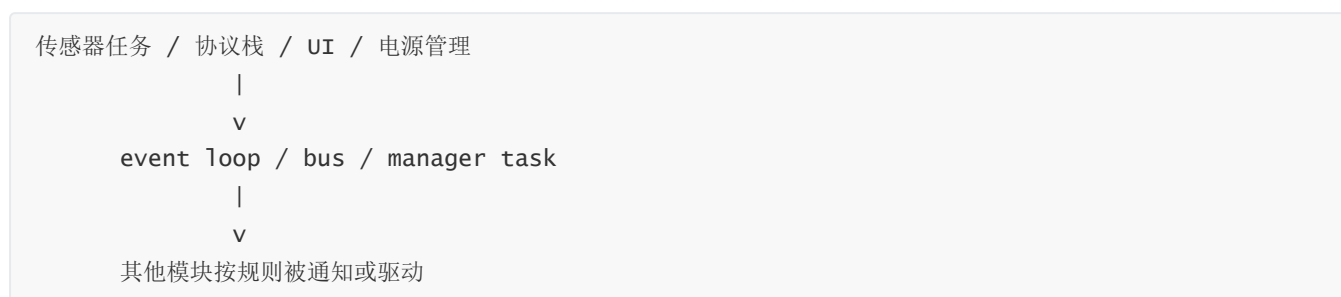
如果系统真正难维护的部分，是模块之间怎么配合，而不是单个模块怎么做事，中介者模式通常就很有价值。

四、结构（Structure）

从嵌入式 C 的角度，可把它看成：



更贴近工程落地时，常见结构是：



这里的关键转变是：

- 原来是模块与模块直接互调
- 现在是模块只与中介者打交道

网络状依赖被收敛成星型依赖。

五、参与者（Participants）

1. Mediator（中介者接口）

它定义统一协调方式。

在嵌入式里，可能体现为：

- 事件发布接口
- 消息投递接口
- 模块协作回调入口
- 统一调度器接口

2. ConcreteMediator（具体中介者）

它是真正的协调中心，负责：

- 保存观察者 / 订阅者关系
- 根据事件决定要通知谁
- 在需要时串行化执行
- 维持某些全局协作状态

它不是简单转发器，而是交互规则真正集中的地方。

3. Colleague（同事模块）

这些模块各自完成自己的职责：

- 采样
- 通信
- 显示
- 存储
- 告警

但它们不直接知道其他模块的细节。

它们只知道如何向中介者上报或订阅。

4. Message / Event / Context（消息与上下文）

嵌入式里的中介者通常离不开消息模型。

典型内容包括：

- 事件类型
- 数据载荷
- 来源模块
- 时间戳
- 状态标志

中介者是否清晰，通常取决于这套消息模型是否清楚。

5. Client（装配者）

Client 负责创建中介者、注册模块、建立观察关系。

在产品代码里，它通常就是系统初始化阶段。

六、协作（Collaborations）

中介者模式的典型执行路径是：

1. 某个模块产生状态变化或事件
2. 该模块不直接调用其他模块，而是通知中介者

3. 中介者根据规则决定：

- 通知谁
- 以什么顺序通知
- 是否同步还是异步
- 是否需要切换上下文

4. 被影响的模块再执行各自动作

因此真正复杂的部分，不再散落在多个模块内部，而是集中在中介者的协作规则中。

这会让系统更容易看懂：

- 模块自己负责什么
- 协调中心负责什么
- 哪些联动是系统级规则

七、效果（Consequences）

收益

1. 降低模块间直接耦合

模块不再互相持有引用，也不需要知道彼此内部实现。

2. 协作规则集中管理

系统联动、跨模块流程、统一调度规则都能在一个地方收敛。

3. 更利于替换与裁剪

把某个模块换成另一实现时，只要它仍遵守中介者协议，其他模块通常不必大改。

4. 更适合控制执行上下文

通过 event loop、bus 或 manager task，可以把跨模块动作拉到受控上下文里，减少乱序调用。

代价

1. 中介者可能膨胀

如果所有逻辑都堆进去，中介者会退化成“上帝模块”。

2. 复杂性只是被集中，不是消失

系统协作仍然复杂，只是从分散复杂改成集中复杂。

3. 简单系统未必值得引入

如果模块很少、关系很简单，直接调用反而更轻。

八、实现要点 (Implementation)

1. 先设计消息边界

中介者不应该转发一堆零散全局变量，而应该有清晰的事件或消息定义。

2. 明确同步与异步策略

有些通知可以同步调用，有些必须排队延后。
这在实时系统里非常关键。

3. 不要让同事模块绕过中介者

如果模块一边通过总线协作，一边又偷偷彼此直调，那么耦合问题会重新长回来。

4. 中介者只负责协作，不抢业务主体职责

它应该负责：

- 路由
- 协调
- 串行化
- 联动规则

而不应吞掉所有业务实现。

5. 注意消息所有权和生命周期

尤其在 C 里，要明确：

- 发布时是复制消息还是传引用
- 观察者读到的是快照还是共享区
- ISR 能否直接发布
- 消息在何时失效

九、典型应用

下面这些都是很多开源项目和厂家 SDK 默认会给出的组织方式：

1. ESP-IDF Event Loop

ESP-IDF 的事件循环允许组件声明事件、注册处理器，并把处理动作放到事件循环这一桥梁上完成。
它强调的是松耦合组件协作，而不是模块之间互相直调。

2. Zephyr zbus

Zephyr zbus 通过 channel 与 observer 组织线程、回调和消息传递，让模块通过软件总线协作。这是非常典型的中介者式架构。

3. 无线/网络 SDK 的中央事件分发

很多 Wi-Fi、BLE、蜂窝模组 SDK 都默认提供 central event handler、event loop 或 manager task。原因很简单：连接状态、IP 状态、扫描结果、重连动作这些联动如果分散到各模块互调，会很快失控。

4. 应用层中心状态管理器

在大量量产产品里，传感器、告警、显示、蜂鸣器、上报模块不会彼此直接纠缠，而是由中心管理器根据事件统一协调。

这也是中介者模式在工程里非常常见的落点。

总结

中介者模式在嵌入式 C 里的本质，就是把模块之间的直接网状协作，收拢到一个统一协调中心，让模块只关心自己的职责，不直接彼此缠绕。



设计模式

面向嵌入式软件开发



针对C语言及嵌入式软件开发的
伴读教程，陪伴学习设计模式这一
进阶知识点。

备忘录模式（Memento）

一、意图（Intent）

在不把模块内部细节暴露给外部的前提下，把某一时刻可恢复的状态保存下来，并在需要时恢复。

站在嵌入式 C 的角度看，备忘录模式控制的不是“对象历史”，而是：

- 某个模块的**可恢复状态**
- 这些状态的**快照边界**
- 快照的**保存时机与恢复时机**
- 快照与模块内部实现之间的**封装边界**

一句话记住它：

模块自己决定哪些状态值得保存，外部只负责触发保存和恢复。

二、动机（Motivation）

在嵌入式系统里，经常有这样一类状态：

- 出厂校准参数
- 用户配置参数
- 通信协议上下文的阶段性状态
- 电机、传感器、显示模块的运行配置

- 升级前需要回滚的关键设置

最直接的做法往往是：

- 业务层直接读取模块内部变量
- 自己拼结构体写入 Flash / NVS / EEPROM
- 恢复时再手工把每个字段写回去

这样的问题很典型：

1. 状态边界会失控

外部代码一旦直接碰模块内部字段，就很容易分不清：

- 哪些字段需要持久化
- 哪些字段只是运行期缓存
- 哪些字段必须成组恢复
- 哪些字段恢复后还要重新 commit

2. 容易破坏模块封装

保存逻辑本来应该知道“哪些状态是有效快照”，结果却变成应用层知道内部布局、知道字段依赖、知道恢复顺序。

3. 非易失存储代价高

嵌入式里写 Flash / NVS 不是零成本操作：

- 有擦写寿命
- 有对齐限制
- 有掉电风险
- 有保存时延
- 还可能受分区、页大小、磨损均衡影响

4. 恢复动作往往不是简单 memcpy

有些参数恢复以后，还需要：

- 重新初始化外设
- 重新装载滤波器系数
- 重新配置通信栈
- 等所有依赖项加载完再统一生效

所以备忘录模式在嵌入式里解决的不是“记录历史”这么抽象的问题，而是：

把“可恢复状态”的定义、导出、恢复，收口到模块自身内部。

三、适用性（Applicability）

下面这些条件出现时，适合考虑备忘录模式：

- 模块内部**有明确状态**，且这些状态需要恢复
- 外部**不能直接依赖内部字段布局**
- 状态保存到非易失介质**代价较高**
- 恢复后可能还要执行**二次生效动作**
- 希望支持“恢复默认值 / 回滚上次配置 / 上电重载参数”

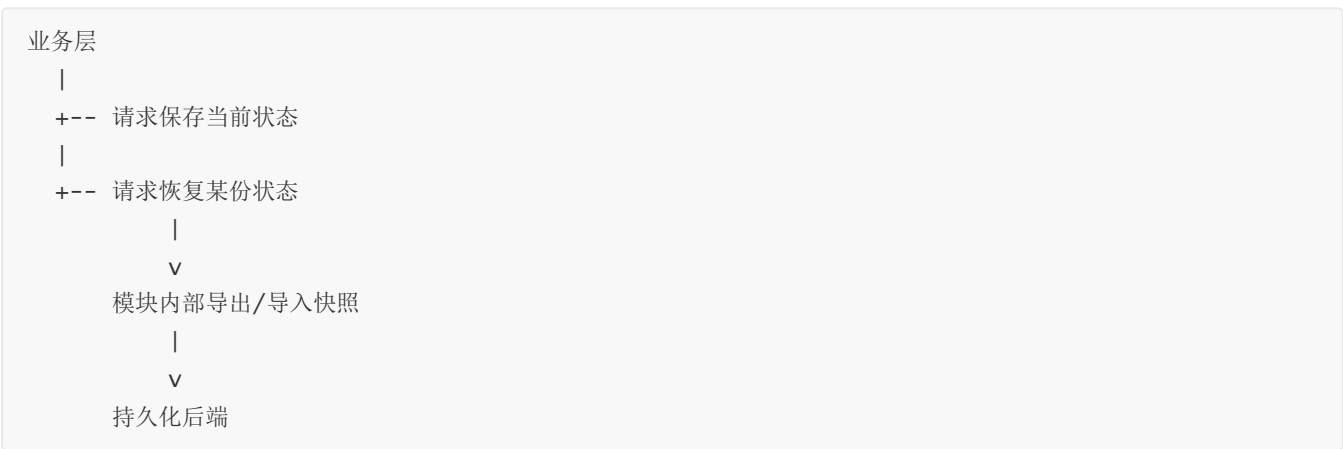
典型嵌入式场景包括：

- 参数子系统
- 校准数据管理
- 升级失败后的配置回滚
- 蓝牙 / Wi-Fi / 网络栈的持久配置
- 复杂外设的 profile 切换与恢复

四、结构（Structure）



也可以按嵌入式流程理解成：



这里最重要的是：

- **业务层不直接改内部布局**

- 快照内容由模块决定
- 存储后端只负责存，不负责理解业务状态

五、参与者（Participants）

1. Originator（发起者 / 状态拥有者）

它是真正持有内部状态的模块。

在嵌入式 C 里，往往就是某个子系统上下文，例如：

- 参数模块上下文
- 电机控制上下文
- 协议会话上下文
- 设备 profile 上下文

它负责：

- 定义哪些状态能进入快照
- 生成快照
- 从快照恢复
- 恢复后做必要的 re-init / commit

2. Memento（备忘录 / 快照）

它不是业务逻辑模块，只是状态载体。

在 C 里通常体现为：

- 一个快照结构体
- 一段序列化 buffer
- 一组键值对导出结果

关键不是“长得像结构体”，而是：

它代表某一时刻的可恢复状态边界。

3. Caretaker（管理者）

它只负责：

- 什么时候保存
- 保存到哪里
- 什么时候恢复
- 保留几份历史快照

它不应该理解模块内部字段语义。

在嵌入式里，通常可以是：

- 参数管理器
- 升级回滚管理器
- 设置子系统
- 配置服务层

六、协作（Collaborations）

整个过程可以拆成两条链：保存链和恢复链。

1. 保存链

1. 业务层判断当前需要保存
2. Caretaker 向模块请求导出快照
3. Originator 生成合法快照
4. Caretaker 把快照写入 Flash / NVS / 文件系统

2. 恢复链

1. 上电或回滚时，Caretaker 从存储介质取回快照
2. 快照交还给 Originator
3. Originator 校验并恢复内部状态
4. 必要时统一 commit，使状态真正生效

最关键的一点是：

是否允许恢复、恢复哪些字段、恢复后还要做什么，都由状态拥有模块说了算。

七、效果（Consequences）

收益

1. 状态边界更清楚

哪些数据是“可恢复状态”，被集中定义，不再散落在应用层。

2. 降低外部耦合

应用层不用知道模块内部字段布局，也不用关心恢复顺序。

3. 更适合持久化与回滚

保存、恢复、恢复默认值、升级回滚，都有统一接口和统一边界。

4. 便于做延迟生效

很多设置不是读到一个字段就立刻生效，而是全部加载完成后统一 commit。

代价

1. 要额外维护快照格式

快照结构、版本兼容、校验字段，都需要设计。

2. 会增加存储与拷贝开销

快照越大，RAM、Flash、时间开销越明显。

3. 版本演进更复杂

字段变化、固件升级、兼容旧数据，都要处理。

八、实现要点 (Implementation)

1. 先划清“哪些状态值得保存”

在嵌入式里，不是所有内部字段都该放进备忘录。

通常要区分：

- **持久配置**：应该保存
- **派生缓存**：恢复后可重算，不必保存
- **瞬时运行态**：通常不保存
- **硬件寄存器镜像**：多半不直接保存，应恢复成配置后再下发

2. 快照格式要带版本与校验

尤其是固件升级会带来结构变化时，要考虑：

- 版本号
- 长度
- CRC / checksum
- 默认值兜底策略

3. 恢复不要等同于裸 memcpy

很多情况下正确顺序应是：

- 先载入原始配置
- 做范围检查
- 解决字段依赖
- 最后统一应用到驱动或硬件

4. 保存策略要考虑写寿命

写 Flash 不能每次字段变化都立刻落盘。

更稳妥的做法往往是：

- dirty 标记
- 定时合并保存
- 关键事件触发保存
- 子树保存而不是全量保存

5. 初始化时机必须明确

恢复逻辑要放在正确阶段：

- 存储后端已就绪
- 依赖的设备已初始化
- 恢复后需要 commit 的模块已经可用

九、典型应用

1. 保存单个配置项或配置子树

在很多实际项目里，并不是每次都全量保存，而是按模块、按 key、按子树保存。Zephyr 文档里提供了 `settings_save_one()`、`settings_save_subtree()` 这类接口，这正符合“快照边界受控”的做法。

2. 上电恢复校准与用户设置

传感器 offset、PID 参数、阈值表、网络凭据，通常都不会让应用层直接拼 Flash 结构体，而是由所属模块自己导出和恢复。

3. 升级回滚前保存旧配置

Bootloader / OTA 管理器在切换镜像之前保存关键配置，失败后再恢复，是备忘录的常见变体。

总结

备忘录模式在嵌入式 C 里，讲的不是“记住过去”，而是把模块自己的可恢复状态封装成可保存、可校验、可回放的快照。



设计模式

面向嵌入式软件开发



针对C语言及嵌入式软件开发的
伴读教程，陪伴学习设计模式这一
进阶知识点。

观察者模式（Observer）

一、意图（Intent）

定义对象之间的一对多依赖关系，使一个对象状态变化时，所有相关方都能收到通知并自行处理。

站在嵌入式 C 的角度，这个模式控制的是：

- 谁发布状态变化
- 谁订阅这类变化
- 通知在什么上下文里执行
- 发布者是否需要知道具体接收者

一句话记住它：

事件源只负责发通知，不负责点名谁来处理。

二、动机（Motivation）

在嵌入式系统里，很多状态变化都会牵动多个模块：

- 按键变化要通知 UI、功耗管理、业务逻辑
- 网络连上以后要通知应用、时间同步、云端上报
- 传感器数据就绪要通知控制回路、记录模块、调试输出
- 电源状态变化要通知显示、充电管理、限频策略

最原始的写法通常是：

- 状态源直接调用多个模块
- 谁要响应，就在状态源里加一个回调
- 模块越来越多时，状态源变成一个分发中心

这会带来几个很典型的问题：

1. 发布者耦合过多接收者

状态源本来只该关注“状态是否变化”，结果却知道：

- 哪些模块需要通知
- 通知顺序
- 某些模块是否允许阻塞
- 某些模块是不是只能在线程上下文处理

2. 新增需求会不断改动源模块

每多一个消费者，状态源就得加注册项、加回调、加 if 分支。

3. 时序和上下文容易混乱

在嵌入式里，通知是不能脱离执行上下文谈的：

- ISR 能不能直接通知？
- 回调是否允许阻塞？
- 是否要切到 work queue？
- 一个慢订阅者会不会拖垮整个发布链？

所以观察者模式在嵌入式里最重要的价值不是“有回调”，而是：

把状态发布与后续响应解耦，并明确通知机制的上下文规则。

三、适用性（Applicability）

适合使用观察者模式的条件通常有：

- 某个状态变化需要通知**多个模块**
- 发布者**不应该知道**所有接收者细节
- 接收者集合可能**增加或裁剪**
- 希望把业务逻辑做成**订阅式拼装**
- 需要明确同步通知、异步通知、ISR 通知等规则

常见场景：

- 事件总线
- 网络状态变化

- 传感器数据就绪通知
- 电源与系统模式切换通知
- GUI / 输入子系统消息广播

四、结构（Structure）

```
事件源 / Subject
|
+----> Observer A
+----> Observer B
+----> Observer C
```

在嵌入式 C 里更常见的写法是：

```
发布者
|
v
事件通道 / 通知中心 / 观察者列表
|
+-- 监听器回调
+-- 线程订阅者
+-- 异步工作队列处理器
```

也就是说，很多工程不会硬做成“一个对象 + 一组对象”，而是做成：

- 一个事件源
- 一个主题 / 通道 / bus
- 多个订阅者

五、参与者（Participants）

1. Subject（主题 / 发布者）

负责：

- 维护观察者关系
- 在状态变化时发布通知

在嵌入式里，它可能是：

- 网络管理器
- 按键模块
- 传感器采集模块
- 电源管理模块
- 总线事件通道

2. Observer（观察者 / 订阅者）

负责接收通知并作出响应。

在 C 里通常体现为：

- 回调函数
- 订阅线程
- work item 处理函数

3. ConcreteObserver（具体观察者）

具体实现各自处理逻辑。

例如：

- UI 更新
- 日志记录
- 控制策略切换
- 故障上报

六、协作（Collaborations）

整体过程通常是：

1. 观察者先注册到某个主题 / 通道
2. 主题状态变化或接收到事件
3. 主题向所有观察者发通知
4. 每个观察者在自己的处理逻辑里响应

这里最关键的不是“通知出去”本身，而是：

- 是同步还是异步
- 是直接在发布上下文执行还是延后执行
- 是否允许丢事件
- 是否有缓冲区和背压

所以在嵌入式里，观察者模式必须和**执行上下文**一起讲。

七、效果（Consequences）

收益

1. 发布者与处理者解耦

事件源不需要知道所有消费方，也不需要直接依赖它们。

2. 系统扩展更自然

新增一个观察者，通常只需注册，不必改事件源主逻辑。

3. 适合裁剪式产品线

某些订阅者开配置即可编译进来，不开则完全不参与。

4. 更适合做多模块拼装

输入、功耗、连接状态这类公共事件，很适合做成统一通知面。

代价

1. 调试链路会变长

事件是谁发的、谁收了、谁又触发了下一步动作，跟踪成本会上升。

2. 慢观察者可能拖慢系统

如果采用同步通知，某个处理者执行过慢会影响整个发布路径。

3. 需要严格定义上下文规则

ISR、线程、work queue、锁持有状态，都必须界定清楚。

八、实现要点 (Implementation)

1. 明确通知上下文

在嵌入式里，第一件事不是定义事件结构体，而是先回答：

- ISR 能否发布？
- 观察者回调在哪执行？
- 是否允许阻塞？
- 是否允许再次发布事件？

2. 数据传递要考虑拷贝和所有权

常见方式有：

- 只发 event id
- 发轻量结构体副本
- 发共享内存引用
- 发队列消息

不同做法会影响：

- RAM 占用
- 生命周期管理
- 并发风险

3. 注册与注销必须安全

动态注册、静态注册、启动期注册，行为要一致。

尤其要处理：

- 遍历时注销
- 回调中再次注册
- 模块卸载后悬挂指针

4. 事件粒度不要失控

事件太细，系统会到处广播；

事件太粗，观察者又要大量二次判断。

九、典型应用

1. Zephyr zbus

Zephyr 的 zbus 明确提供 channel 与 observer 的组织方式，支持 many-to-many 通信，并区分 listener、async listener、subscriber 等观察者形态。发布和读取操作还说明了其可在 RTOS 线程上下文甚至 ISR 中使用，这正是嵌入式观察者模式最关键的“上下文规则”。

2. ESP-IDF 默认事件循环

ESP-IDF 的 `esp_event` 把系统事件投递到默认事件循环，组件和应用通过注册 handler 来接收 Wi-Fi、IP 等事件；默认事件循环的 handle 对用户隐藏，应用面对的是“注册处理者”，不是直接耦合到事件源。

3. 网络、功耗、输入子系统事件广播

这类模块几乎都天然是一源多收：

- 一个连接状态变化，多个模块都要感知
- 一个低电量事件，多个策略都要调整
- 一个按键动作，多个上层功能都可能响应

总结

观察者模式在嵌入式 C 里，就是把“状态变化后的多方响应”从发布者身上拆下来，做成可订阅、可裁剪、可控上下文的通知机制。



设计模式

面向嵌入式软件开发



针对C语言及嵌入式软件开发的
伴读教程，陪伴学习设计模式这一
进阶知识点。

状态模式（State）

一、意图（Intent）

允许一个对象在其内部状态改变时改变它的行为，使其看起来像修改了所属类别。

站在嵌入式 C 的角度，可以把这句话改写成：

把“当前处于什么状态”与“当前应该怎么响应事件”绑定起来。

状态模式真正控制的是：

- 当前状态的归属位置
- 状态切换的合法路径
- 不同状态下对同一事件的不同处理方式
- entry / run / exit 的执行时机

一句话记住它：

不是 if-else 多，而是“行为由当前状态决定”。

二、动机（Motivation）

嵌入式系统里有大量天然的状态机对象：

- 设备初始化流程
- 蓝牙 / Wi-Fi / TCP 连接过程

- 充电状态
- 电机控制阶段
- UI 页面流程
- OTA 升级过程

最常见的坏味道是：

```
if (ctx->state == INIT) { ... }  
else if (ctx->state == IDLE) { ... }  
else if (ctx->state == RUNNING) { ... }  
else if (ctx->state == ERROR) { ... }
```

然后在多个函数里都重复判断：

- 收到命令时判断一次
- 定时器到时判断一次
- 中断事件来时判断一次
- 超时恢复时再判断一次

这样会带来几个问题：

1. 状态判断到处散落

状态本来应该集中管理，结果变成整个系统都在问 `ctx->state`。

2. 切换规则不集中

从哪个状态能切到哪个状态、切换时要做什么，容易散落到多个函数里。

3. entry / exit 动作容易漏

很多硬件相关逻辑不是“改个枚举值”就结束，而是要：

- 进入状态时打开资源
- 退出状态时释放资源
- 切换时重置计数器 / 超时器 / 标志位

4. 状态与行为没有绑定

同一个事件在不同状态下处理方式不同，但代码组织却没有体现这一点。

所以状态模式在嵌入式里最重要的价值是：

把状态、事件处理、切换逻辑和出入状态动作收口到统一结构里。

三、适用性 (Applicability)

适合考虑状态模式的条件有：

- 模块有明确的有限状态

- 同一事件在不同状态下**处理不同**
- 状态切换有严格规则
- 进入和退出状态时要执行动作
- 业务层不希望到处直接判断状态值

典型场景：

- 通信连接管理
- 设备生命周期管理
- 任务流程控制
- 充电 / 休眠 / 唤醒
- 升级与恢复流程

四、结构（Structure）

```
应用层事件
  |
  v
Context（状态上下文） ----> Current State
                                |
                                +-- entry()
                                +-- run()/handle_event()
                                +-- exit()
```

从嵌入式执行路径看，可以理解为：

```
事件 / 定时器 / 轮询
  |
  v
状态机上下文 ctx
  |
  v
当前状态处理函数
  |
  +-- 保持当前状态
  +-- 切换到新状态
  +-- 终止 / 错误 / 上报
```

五、参与者（Participants）

1. Context（状态上下文）

它保存：

- 当前状态
- 状态机公共数据

- 与状态切换相关的计数器、标志位、资源句柄

在嵌入式 C 里，它通常就是一个上下文结构体。

2. State（抽象状态）

定义统一的状态处理接口。

如果做得更工程化，通常会拆成：

- `entry`
- `run` 或 `handle_event`
- `exit`

3. ConcreteState（具体状态）

每个具体状态负责：

- 本状态下如何处理事件
- 何时跳转到新状态
- 进入和退出时做什么动作

六、协作（Collaborations）

通常的执行过程是：

1. 初始化状态机上下文
2. 设定初始状态
3. 事件或调度到来时，交给当前状态处理
4. 当前状态决定：
 - 自己处理完
 - 向上层传播
 - 切换到新状态
5. 若切换，则执行 `exit -> transition -> entry`

状态模式的关键不是“有状态枚举”，而是：

当前状态自己决定当前事件如何被解释。

七、效果（Consequences）

收益

1. 状态逻辑集中

每个状态的行为和切换规则更容易放在一起看。

2. 减少散落的条件分支

不用在每个业务函数里都反复问“当前 state 是什么”。

3. 更适合复杂生命周期管理

尤其适合初始化、连接、异常、恢复这类有阶段边界的流程。

4. entry / exit 语义清晰

资源申请、资源释放、计时器重置、日志上报，都能放到明确位置。

代价

1. 状态数多时结构会变大

状态越细，函数与表项越多。

2. 需要更严格的切换规则设计

否则状态会爆炸，或者出现非法跳转。

3. 调试需要观察状态轨迹

单看某个函数常常不够，要看“从哪里来、往哪里去”。

八、实现要点 (Implementation)

1. 状态上下文要集中

不要把状态相关数据散落到多个全局变量里。

常见做法是：

- 一个 `ctx`
- 里面包含当前状态
- 再加共享标志位、计时器、错误码等

2. 区分“状态”和“事件”

状态是系统当前阶段，事件是触发处理的输入。

两者不能混成一个大枚举。

3. entry / exit 中放资源边界动作

例如：

- 打开 / 关闭外设
- 启停超时计时器
- 清空或冻结缓冲区
- 设置 busy / ready / error 标志

4. 非法跳转要显式防御

嵌入式里状态机常涉及硬件时序，非法跳转不能默默放过。

5. 层级状态机要谨慎使用

它很强，但如果团队还在频繁改业务流程，先把平面状态机理顺更重要。

九、典型应用

1. 连接管理

例如：

- DISCONNECTED
- CONNECTING
- CONNECTED
- RETRY_BACKOFF
- ERROR

相同的“收到数据”“连接超时”“用户断开”事件，在不同状态下意义完全不同。

2. 电源 / 充电 / 升级流程

这类流程都有强状态边界和明显的 entry / exit 动作，非常适合状态模式。

总结

状态模式在嵌入式 C 里，本质上就是把“当前阶段”和“当前处理规则”绑在一起，让行为随状态切换而切换。



设计模式

面向嵌入式软件开发



针对C语言及嵌入式软件开发的
伴读教程，陪伴学习设计模式这一
进阶知识点。

策略模式（Strategy）

一、意图（Intent）

定义一系列算法，把它们一个个封装起来，并且使它们可以相互替换。

在嵌入式 C 里，这句话更应该理解成：

上层面对的是同一类能力接口，至于底下具体采用哪套实现策略，由运行时或编译期装配决定。

这个模式控制的是：

- 同一功能的多种实现选择
- 算法 / 驱动后端的替换边界
- 上层对下层策略的依赖方式
- 选择策略的时机（编译期 / 初始化期 / 运行期）

一句话记住它：

上层只说“我要做这件事”，至于用哪种做法，由策略层决定。

二、动机（Motivation）

嵌入式系统里，“同一个动作有多种实现”是很常见的：

- 不同芯片的 UART / SPI / I2C 驱动实现不同
- 存储后端可能是 NOR Flash、NAND、EEPROM、NVS

- 过滤算法可能有轻量版、精确版、省电版
- 日志输出可能走 UART、RTT、Flash、网络
- 通信重试策略可能有快速重试、指数退避、静默失败

最直接的坏味道通常是：

- 上层充满 `if (chip_xxx)`
- 上层自己判断是否使用某种算法
- 每增加一个平台就多一层分支

问题会越来越明显：

1. 上层被具体实现污染

业务层本来只想“发一个字节”“存一个配置”“输出一条日志”，结果却必须知道底层是哪家芯片、哪种介质、哪条总线。

2. 替换成本高

新增一种实现，就要改上层调用路径。

3. 无法形成统一能力接口

系统明明做的是同一类事情，却没有稳定的抽象边界。

所以策略模式在嵌入式里真正解决的是：

把“要做什么”和“具体怎么做”隔离开，让实现策略可替换。

三、适用性（Applicability）

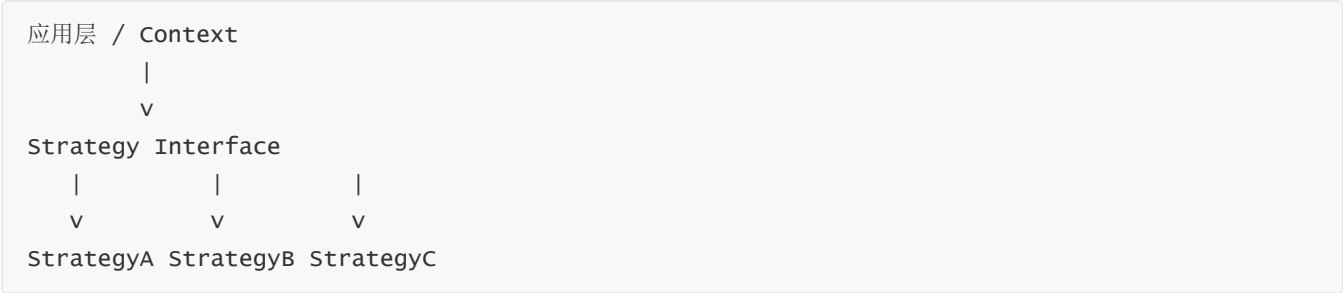
适合使用策略模式的条件包括：

- 同一个能力存在多种实现方案
- 上层只关心统一行为，不应关心具体实现细节
- 不同产品 / 板级 / 芯片需要切换实现
- 希望通过函数表或接口表完成解耦
- 算法选择或驱动选择可能随配置变化

常见场景：

- 驱动后端抽象
 - 存储后端抽象
 - 编解码算法切换
 - 过滤 / 控制算法切换
 - 日志 / 网络 / 文件系统后端切换
-

四、结构（Structure）



在 C 里通常长这样：

```
ctx
|
+-- 指向一张 ops 表
    |
    +-- init()
    +-- read()
    +-- write()
    +-- control()
```

所以策略模式在嵌入式 C 里的典型落地，不是类继承，而是：

- 一份统一接口定义
- 多个实现函数集
- 一个函数指针表 / API 表
- 上层持有当前策略引用

五、参与者（Participants）

1. Context（上下文）

负责：

- 持有当前策略
- 在合适的时候调用策略接口
- 保存与策略无关但与业务有关的上下文数据

2. Strategy（策略接口）

定义统一能力边界。

在 C 中通常体现为：

- 一组函数原型
- 一个 API 结构体
- 一套约定好的返回值语义

3. ConcreteStrategy（具体策略）

实现某一套具体行为。

比如：

- 不同芯片驱动后端
- 不同存储介质后端
- 不同滤波算法
- 不同重试策略

六、协作（Collaborations）

整体流程很简单：

1. 系统初始化时选定某个策略
2. Context 保存该策略引用
3. 上层调用统一接口
4. 实际行为落到对应策略实现

要点在于：

- 调用方不再写平台分支
- 切换实现时尽量不改业务层
- 策略可以按产品线装配

七、效果（Consequences）

收益

1. 统一上层调用面

应用层看到的是稳定接口，而不是五花八门的底层实现。

2. 更利于产品线扩展

同一套业务代码可以挂接不同策略实现。

3. 替换实现更自然

新增平台或新增算法时，影响主要落在策略实现层。

4. 适合函数表风格的 C 工程

这种模式和嵌入式里广泛使用的 `ops / api / driver table` 非常契合。

代价

1. 多一层间接调用

会带来一点点指针跳转和理解成本。

2. 接口设计必须稳定

策略接口一旦设计不好，后续所有实现都会受影响。

3. 简单场景可能显得偏重

如果永远只有一种实现，引入策略层可能没有必要。

八、实现要点 (Implementation)

1. 接口只放“共同能力”

不要把某个特定平台的私有能力硬塞进公共接口。

2. 明确上下文所有权

策略实现是否持有设备私有数据、由谁初始化、生命周期归谁，都要说清。

3. 区分编译期选择和运行期选择

嵌入式里很多“可替换”其实是在初始化时就定死了，不一定真的支持热切换。

4. 返回值和错误语义要统一

否则上层虽然接口统一了，错误处理还是散的。

5. 控制私有扩展口

若必须支持某些后端私有扩展，最好通过单独扩展接口而不是污染主接口。

九、典型应用

1. 同一功能的多后端存储

例如设置保存可以落到不同存储介质；上层只关心“保存配置”，不关心底下到底是 NVS、FCB 还是文件系统。Zephyr settings 也明确提供多种后端实现。□cite□turn609390search3□

2. 多种算法切换

例如电机控制里切换不同控制律、滤波模块切换不同滤波器、通信模块切换不同重传策略。

总结

策略模式在嵌入式 C 里，最常见的落地就是“统一接口 + 多套 ops 表实现”，让业务层不再被具体驱动或算法绑死。



设计模式

面向嵌入式软件开发



针对C语言及嵌入式软件开发的
伴读教程，陪伴学习设计模式这一
进阶知识点。

模板方法模式（Template Method）

一、意图（Intent）

定义一个操作中的算法骨架，而将某些步骤延迟到子类中，使得子类可以不改变算法结构即可重定义算法的某些步骤。

改成嵌入式 C 的话，更好理解成：

框架先把主流程固定下来，再把少数可变步骤留成钩子、回调或可替换步骤。

它控制的是：

- 流程骨架由谁决定
- 哪些步骤固定，哪些步骤允许变化
- 用户代码能插手到流程的哪个位置
- 插件 / 回调能否破坏主时序

一句话记住它：

主流程归框架，变化点归钩子。

二、动机（Motivation）

在嵌入式工程里，有很多流程是不能让业务层随便改顺序的：

- 网络服务接收请求的总体流程

- 配置加载后的整体生效流程
- 驱动初始化的总体阶段顺序
- 一次事务处理的固定阶段
- 一次协议包处理的解析、校验、执行、回包流程

如果不固定骨架，常见问题是：

1. 每个模块都自己拼流程

结果同一类流程在不同模块里顺序不同、错误处理不同、资源释放也不同。

2. 关键前后置动作容易漏

例如：

- 没有先校验就执行业务
- 资源没申请完就开始处理
- 出错时忘记收尾
- 某些步骤必须在统一线程上下文执行，却被乱放到别处

3. 业务代码容易越权改框架时序

框架本来应该保证主路径稳定，结果用户代码想怎么插就怎么插。

模板方法模式的核心价值就是：

先把不可乱动的流程骨架固定住，再开放有限的变化点。

三、适用性（Applicability）

适合使用模板方法模式的条件包括：

- 主流程必须稳定
- 不同产品或模块只在少数步骤上变化
- 某些前后置动作必须被统一保证
- 希望用户只实现钩子，不重写整个流程
- 需要框架统一控制错误处理和资源收尾

常见场景：

- 服务器 / 协议处理框架
 - 配置加载与提交流程
 - 驱动初始化框架
 - 测试步骤框架
 - 请求解析与分发框架
-

四、结构（Structure）

Framework Skeleton

```
|  
+-- step A (fixed)  
+-- step B (hook)  
+-- step C (fixed)  
+-- step D (hook)
```

在 C 里更常见的表现是：

固定主函数

```
|  
+-- 前置检查  
+-- 调用用户回调/钩子  
+-- 后置清理  
+-- 统一返回
```

也就是说，模板方法在嵌入式 C 中不一定依赖“继承”，更常见的是：

- 固定框架流程
- 若干可注册回调
- 若干可选钩子函数
- 严格的调用顺序

五、参与者（Participants）

1. AbstractClass / Framework Skeleton（框架骨架）

负责：

- 定义整体处理顺序
- 固定不可变步骤
- 在预定位置调用变化点
- 保证错误处理与收尾逻辑

2. Primitive Operations / Hooks（原语步骤 / 钩子）

这些是留给用户实现或注册的可变步骤。

在 C 中通常体现为：

- 回调函数
- ops 中的某个钩子
- 弱符号回调
- 注册表中的处理函数

3. Concrete Implementation（具体实现）

只负责补上变化步骤，不掌控整个框架流程。

六、协作（Collaborations）

模板方法模式的典型执行过程是：

- 1. 框架入口函数被调用
- 2. 框架先执行固定的前置步骤
- 3. 在指定位置调用用户钩子
- 4. 钩子返回后，框架继续完成后续步骤
- 5. 最后统一做清理、返回状态

这里最重要的是：

用户实现的是步骤，不是整个时序。

七、效果（Consequences）

收益

1. 主流程稳定

关键时序、资源边界、收尾逻辑都由框架保证。

2. 变化点集中

用户只实现被允许变化的步骤，职责边界更清楚。

3. 更利于统一错误处理

前置检查、异常返回、释放资源可以收口在框架层。

4. 适合做平台与业务解耦

平台层负责骨架，业务层负责少量回调实现。

代价

1. 钩子设计不好会束手束脚

开放点太少，用户不够用；开放点太多，又会把框架打散。

2. 流程调试要同时看框架和钩子

有时候逻辑问题不在钩子本身，而在钩子被调用的阶段。

3. 过度模板化会增加理解门槛

简单流程若硬做成模板方法，反而不如直接函数调用清楚。

八、实现要点 (Implementation)

1. 先分清“骨架步骤”和“变化步骤”

不要把所有步骤都做成钩子。真正需要固定的往往是：

- 参数检查
- 上下文建立
- 加锁 / 解锁
- 资源申请 / 释放
- 错误收尾

2. 钩子执行上下文必须明确

要说明：

- 钩子在哪个线程执行
- 是否允许阻塞
- 是否允许再次调用框架 API
- 回调失败后框架如何处理

3. 钩子返回值语义要统一

不能让每个模块自己解释成功、失败、继续还是中断。

4. 不要让钩子破坏骨架资源约束

例如框架持锁时调用钩子，就必须明确钩子不能做哪些动作。

九、典型应用

1. ESP-IDF HTTP Server

ESP-IDF 的 HTTP server 明确把服务器总体流程固定在框架中：`httpd_start()` 创建服务器和任务，任务循环解析请求并根据 URI 匹配已注册 handler，再由用户提供的 URI handler 处理具体业务。也就是说，请求处理骨架由框架决定，业务只填自己的处理步骤。□cite□turn262787search1□turn262787search2□turn262787search5□

2. 配置加载后的统一提交流程

Zephyr settings 中，加载流程会先调用各项的 `h_set`，在全部加载完成后再统一调用 `h_commit`。这正体现了“整体流程固定，局部处理留给模块自身”的模板方法思想。

3. 厂家 HAL 中断回调骨架

大量 MCU 厂家 SDK 都采用“固定 IRQ 处理骨架 + 用户回调”的组织方式，本质上也是模板方法，只是用 C 回调或弱符号表达，而不是类继承。

总结

模板方法模式在嵌入式 C 里，就是先把不可乱动的流程骨架固定住，再把少数业务差异留成受控钩子。



设计模式

面向嵌入式软件开发



针对C语言及嵌入式软件开发的
伴读教程，陪伴学习设计模式这一
进阶知识点。

访问者模式（Visitor）

一、意图（Intent）

表示一个作用于某对象结构中的各元素的操作，使你可以在不改变各元素类的前提下定义作用于这些元素的新操作。

翻成嵌入式 C 的表达，更容易理解成：

数据结构保持稳定，而对这批结构执行什么操作，可以在结构外部继续扩展。

它控制的是：

- 元素结构是否稳定
- 新操作应该加在元素内部还是外部
- 遍历与处理职责如何分离
- 新增操作时是否要改原有节点结构

一句话记住它：

节点结构尽量不动，新操作放在外部访问逻辑里追加。

二、动机（Motivation）

访问者模式在纯 MCU 业务代码里不像观察者、状态那样高频，但在下面这类地方非常有价值：

- 配置树 / 设备树遍历
- 协议树 / 报文树处理
- AST / 脚本语法树处理
- 调试导出、统计、校验、生成代码等“多种操作叠加在同一结构上”

最直接的坏味道是：

- 每个节点类型里塞满各种无关操作
- 想增加一种新分析功能，就要修改所有节点处理函数
- 结构本身和操作逻辑强耦合

这在嵌入式工程里尤其麻烦，因为很多“结构”是很稳定的：

- 配置树格式很稳定
- 报文节点格式很稳定
- 设备描述结构很稳定

真正经常变化的，反而是“对它做什么”：

- 导出
- 校验
- 统计
- 打印
- 生成表
- 生成初始化代码

所以访问者模式真正想解决的是：

元素结构不想频繁改，但新操作会持续增加。

三、适用性（Applicability）

适合考虑访问者模式的条件包括：

- 有一组相对稳定的数据结构节点
- 未来会不断增加新的处理操作
- 不希望每次加新操作都改节点定义
- 操作天然依赖“遍历一整棵结构”
- 结构层与处理层需要分离

常见场景：

- 设备树 / 配置树处理
 - 协议树解析后的二次处理
 - 脚本 / 规则树分析
 - 调试打印、校验、统计、生成代码
-

四、结构（Structure）

```
Object Structure
|
+-- Element A
+-- Element B
+-- Element C

Visitor X
Visitor Y
Visitor Z
```

在嵌入式 C 里更常见的落地是：

```
树 / 节点集合
|
+-- 遍历框架
    |
    +-- 访问操作A（打印）
    +-- 访问操作B（校验）
    +-- 访问操作C（导出）
```

也就是说，很多 C 工程不会严格实现 GoF 式“双分派”，
但会实现它的核心意图：

- 节点结构保持稳定
- 新操作以外置访问逻辑增加

五、参与者（Participants）

1. Element（元素）

表示结构中的节点。

在嵌入式里可以是：

- 设备节点
- 配置节点
- 协议字段节点
- 语法树节点

2. Visitor（访问者）

表示一种“要对节点做的操作”。

例如：

- 打印访问者
- 校验访问者

- 统计访问者
- 代码生成访问者

3. Object Structure（对象结构）

保存整组节点并提供遍历入口。

六、协作（Collaborations）

执行过程通常是：

1. 系统拿到一组稳定节点结构
2. 选定某种访问操作
3. 遍历框架按节点顺序访问
4. 访问者对不同节点执行对应逻辑
5. 遍历结束后得到结果

访问者模式的关键不在“有遍历”本身，而在：

■ 新增操作时，不想去改原有节点定义和已有节点代码。

七、效果（Consequences）

收益

1. 新增操作更容易

增加一种新处理方式，主要新增访问逻辑，不必改所有节点结构。

2. 数据结构更稳定

节点职责更聚焦在“表达结构”，而不是承担一大堆无关行为。

3. 适合工具链与分析类任务

例如打印、校验、导出、生成代码，这些操作天然适合外置。

代价

1. 新增节点类型会牵动访问者

访问者模式对“新增操作”友好，但对“新增元素类型”不如前者友好。

2. 在纯 C 中做满配 GoF 形式偏重

很多嵌入式项目会只保留思想，不硬做完整双分派结构。

3. 运行期 MCU 业务里不一定高频

它更常见于配置树、协议树、工具链和生成侧，而不是简单驱动层。

八、实现要点 (Implementation)

1. 先判断你面对的是“稳定结构 + 多操作”还是“多结构 + 少操作”

若节点类型老在变，访问者模式不一定合算。

2. 在 C 中可以用“遍历函数 + 回调 / ops 表”表达核心思想

不一定非要追求类 OO 形式。

3. 让节点结构只保存结构相关数据

不要把打印、校验、导出等逻辑都塞回节点定义里。

4. 访问过程中要注意栈深度与资源限制

若结构是树或递归结构，在 MCU 上要特别关注：

- 递归深度
 - 临时缓冲区
 - 访问过程中的锁与上下文
-

九、典型应用

1. Devicetree / 配置树遍历

Zephyr 提供了一组 devicetree 的“for-each”宏，例如 `DT_FOREACH_CHILD()`、`DT_FOREACH_NODE()`，允许在不改节点描述本身的前提下，对节点集施加不同操作。它不一定是教科书式双分派，但非常符合访问者模式“结构稳定、操作外置”的核心思想。[□cite□turn206503search2□turn206503search3□turn206503search7□](#)

2. 配置校验 / 导出 / 统计

同一棵配置树，可能既要打印、又要做合法性检查、又要生成默认值表，这些操作都适合做成外置访问逻辑。

3. 协议树与脚本树处理

例如调试器、配置编译器、代码生成工具，对同一组节点执行多种分析与导出操作，这是访问者模式最适合发力的地方。

总结

访问者模式在嵌入式 C 里最值得抓住的一点是：当节点结构相对稳定、而新操作还会不断增加时，把这些操作放到结构外部去扩展。